



University of Milano-Bicocca

Department of Informatics, Systems and Communication

Master's Degree Program in Data Science

An Approach for Extracting Training Data from fine-tuned Large Language Models for Code

Supervisor:

Fabio Mercorio

Co-supervisor:

Maliheh Izadi

Master's degree thesis by:

Fabio Salerno

MATR. N. 861419

Academic Year 2023-2024

Contents

1	Introduction	1
1.1	Contributions	3
1.2	Thesis Structure	5
1.3	Reproducibility	6
2	Preliminaries	7
2.1	Language Modeling	7
2.2	Statistical Language Models	8
2.3	Neural Language Models	10
2.3.1	Word Embeddings	11
2.3.2	Recurrent Neural Networks	13
2.4	Pre-trained Language Models	16
2.4.1	The Attention Mechanism	16
2.4.2	The Transformer Architecture	17
2.4.3	Pre-training and Fine-tuning Paradigm	19
2.5	Large Language Models	20
2.5.1	Model Training	21
2.5.2	Model Generation	22
2.5.3	State of The Art Large Language Models	23
2.5.4	Drawbacks and Risks of Large Language Models	26
2.6	Large Language Models for Software Engineering	28
2.6.1	Downstream Tasks in Code Language Models	28
2.6.2	Data in Code Language Models	30

2.6.3	Evaluating Code Language Models	33
2.7	Personal Data Compliance in the European Union	36
2.8	Software Licenses and IP Regulations	40
2.8.1	License Compliance and Code Language Models	42
3	Related Work (SOTA)	45
3.1	Memorization	45
3.2	Memorization in Code Language Models	47
3.3	Membership Inference Attacks	48
3.4	Untargeted Data Extraction Attacks	49
3.5	Targeted Data Extraction Attacks	50
3.6	Memorization after Fine-tuning	51
3.7	Our contribution	53
4	Problem Definition	54
4.1	Code Completion	54
4.2	StarCoder2 and TheStack v2	56
4.3	Research Questions	56
5	Methodology	58
5.1	Fine-Tuning	58
5.1.1	Dataset	58
5.1.2	Training	61
5.2	Data Extraction Attacks	61
5.2.1	Data Extraction Security Game	62
5.2.2	Data Extraction Dataset	63
5.2.3	Performing the Attacks	65
5.3	Analysis through Categorization	65
6	Results	67
6.1	RQ1: Extracting Pre-training Data from Pre-trained Models	67
6.2	RQ2: Extracting Pre-training Data From Fine-tuned Models	68

6.3	RQ3: Extracting Fine-tuning Data	74
6.4	RQ4: Type of Extracted Data	77
7	Discussion	80
7.1	Pre-training Data Memorization - Pre-trained Models	80
7.2	Pre-training Data Memorization - Fine-tuned Models	81
7.3	Fine-tuning Data Memorization	82
7.4	Implications	83
7.5	Recommendations	84
7.6	Ethical considerations	85
7.7	Limitations and Threats to the Validity	85
8	Conclusions	87

Acronyms

AI Artificial Intelligence

BERT Bidirectional Encoder Representations from Transformers

BLEU Bilingual Evaluation Understudy

CLMs Code Language Models

EM Exact Match

GDPR General Data Protection Regulation

GPT Generative Pre-trained Transformers

GRU Gated Recurrent Units

ICTs Information Communication Technologies

IP Intellectual Property

LLMs Large Language Models

LSTM Long Short-Term Memory

MLP Multi-Layer Perceptrons

NIST National Institute of Standards and Technology

NLMs Neural Language Models

NLP Natural Language Processing

PII Personally Identifiable Information

PLMs Pre-trained Language Models

RLHF Reinforcement Learning from Human Feedback

RNNs Recurrent Neural Networks

SE Software Engineering

SLMs Statistical Language Models

SOTA State Of The Art

Acknowledgments

"Nelle corse di cavalli basta mettere il musetto davanti, non c'è bisogno di vincere di 100, musetto davanti, il muso, fotografia corto muso, semplice." Massimiliano Allegri.

"Quanta gente esce dall'università e prende 110 a ripetere e ad imparare a memoria? Poi li metti a lavorare nella vita privata e non capiscono nulla. La vita è fatta di praticità, c'è quello che esce con 60 e che è un genio. Il calcio è la stessa roba." Massimiliano Allegri.

"Napoleone lo sai cosa diceva? Napoleone diceva sempre, meglio avere generali fortunati, che bravi" Antonio Cassano.

Abstract

Code language models, while widely popular, are often trained on unsanitized source code gathered from across the Internet. Previous work revealed that *pre-trained* models can remember the content of their training data and regurgitate them through data extraction attacks. Due to the large size of current models, only a few entities have the resources for pre-training such models. However, fine-tuning requires fewer resources and is increasingly used by both small and large entities for its effectiveness on specialized data. Such small curated data for fine-tuning might contain sensitive information or proprietary assets. In this study, we attack *both* pre-trained and fine-tuned code language models to investigate the extent of data extractability. We first develop a custom benchmark to assess the vulnerability of both pre-training and fine-tuning samples to extraction attacks. Our findings reveal that 54.9% of extractable pre-training data could be retrieved from StarCoder2-15B, whereas this number decreased to 23.5% after fine-tuning. This indicates that fine-tuning reduces the extractability of *pre-training* data. However, compared to larger models, *fine-tuning smaller* models increases their vulnerability to data extraction attacks on *fine-tuning* data. Given the potential sensitivity of fine-tuning data, this can lead to more severe consequences. Lastly, we also manually analyzed 2000 extractable samples before and after fine-tuning. We also found that data carriers and licensing information are the most likely data categories to be memorized from pre-trained and fine-tuned models, while the latter is the most likely to be forgotten after fine-tuning.

Chapter 1

Introduction

Large Language Models (LLMs) have seen widespread adoption across various fields, raising interest in their ability to generate human-like text and accomplish complex tasks. LLMs have become relevant within domains like Software Engineering (SE) due to their capacity to automate coding-related tasks, such as code completion, documentation generation, and bug detection, enhancing productivity and efficiency. [1], [2]. Despite their immense popularity, LLMs for code are commonly trained on *unsanitized* datasets of source code scraped from the Internet. Previous work has revealed that the content of these datasets can be memorized and extracted by attackers through various data extraction techniques [3]–[7]. An adversary can prompt the model to make it regurgitate fragments of its training data. These attacks may reveal sensitive or proprietary information embedded in the training corpus, such as Personally Identifiable Information (PII) like full names, email addresses, phone numbers, or even copyrighted content, including code. Figure 1.1 illustrated an example of a data extraction attack on the StarCoder2-7B model [8], consisting of inputting a precise sequence of text to extort information memorized during training. The attack made the model successfully regurgitate personally identifiable information from the training corpus. Memorization in code models presents significant privacy and security concerns, particularly when these models are deployed in real-world applications. For example, the deployed models for commercial

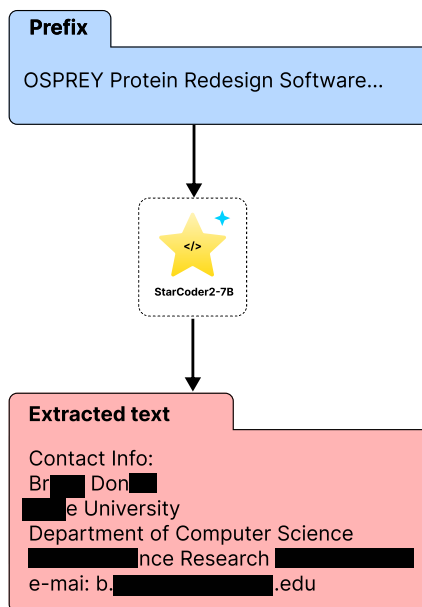


Figure 1.1: Example of a data extraction attack on StarCoder2-7B.

use in GitHub Copilot ¹ and Amazon CodeWhisperer ² can be coaxed to emit hardcoded credentials captured during training, including API keys and OAuth IDs. These credentials, if exposed, may allow unauthorized access to applications, systems, or networks. [9].

Pre-training LLMs is an infrequent but highly resource-intensive process, primarily due to the immense size of these models, which often contain billions or even trillions of parameters. This phase demands substantial computational resources and time, typically necessitating hundreds of thousands of GPUs. For instance, pre-training the smaller version of the LLaMA 2 model [10], which contains 7 billion parameters, required approximately 184,320 GPU hours on NVIDIA A100 GPUs, while the largest version, with 70 billion parameters, demanded around 1,720,320 GPU hours [10]. Although LLaMA 2 was pre-trained using proprietary clusters, it is noteworthy that the cost of an NVIDIA A100 instance on Amazon Web Services (AWS) is approximately \$4.10 per GPU hour ³. This highlights the significant expense associated with pre-training LLMs, making training strategies like fine-tuning an increasingly dominant approach for end-users seeking

¹GitHub Copilot: <https://github.com/features/copilot>

²Amazon CodeWhisperer: <https://docs.aws.amazon.com/codewhisperer/>

³Amazon Web Services: <https://aws.amazon.com>

to customize models for specific tasks.

Fine-tuning is the process of adapting a pre-trained model to a specific task by further training it on a smaller, domain-specific dataset [11]. This process helps the model improve its performance in particular use cases without requiring it to be trained from scratch. However small curated datasets employed for fine-tuning may contain proprietary or sensitive information [5], [12]. This raises concerns about the potential for sensitive data within these datasets to be inadvertently extracted from the fine-tuned models [13]–[15].

1.1 Contributions

To our knowledge, previous studies have investigated data memorization and extraction attacks on pre-trained LLMs for code [5], [14]. However, empirical investigations into the extractability of fine-tuned code models remain limited. This is especially concerning given that fine-tuning, commonly employed to enhance model performance on specific tasks, often involves smaller, curated datasets that can contain sensitive or proprietary information. Understanding the behavior of fine-tuned models is critical for evaluating the risks of data leakage, as the models may inadvertently memorize and leak parts of their training data. In this work, we explore the extent to which fine-tuned LLMs for code leak data from their pre-training and fine-tuning corpora, addressing this gap in current literature.

As there is no comprehensive framework or approach for measuring memorization, we start by defining a data extraction security game. This game is grounded in the theory behind membership inference attacks and k -extractability [3], [16]. These concepts help quantify the likelihood of a model inadvertently revealing training data when presented with specific inputs. Using this security game, we develop a framework to quantify memorization in LLMs, using data extractability as a proxy. While memorization of training data can manifest as non-exact duplication [17], measuring the rate of data extraction provides a lower bound on data leakage in LLMs.

We first develop a custom benchmark to assess the vulnerability of pre-training and

fine-tuning samples to extraction attacks and identify vulnerable samples. We then test the extractability of pre-training and fine-tuning data on the Starcoder2 model family [8]. We chose StarCoder2 as it is one of the most recent open-source code models. Starcoder2 and its derivatives are the best-performing models with both open weights and open data on the EvalPlus Leaderboard [18].⁴ Its open-data nature provides us with complete access to the training corpus, which is essential for crafting the extractability benchmark and conducting targeted attacks. The benchmark allows us to measure and compare the extractability of both pre-training and fine-tuned datasets. To further enrich our analysis, we manually categorize the extracted samples to identify specific types of data, that according to Al-Kaswan *et al.* are particularly exposed to memorization, such as licensing information, proprietary code, and structured data formats like dictionaries or configuration files [5]. This fine-grained categorization not only enables us to quantify the extent of memorization for each data category but also highlights the potential security implications of such memorization. By understanding which kinds of data are more exposed, we aim to better assess the risks of unintended data leakage, providing critical insights into privacy concerns, especially in domains where the protection of intellectual property or sensitive personal information is paramount.

Our key results: *Fine-tuned LLMs for code exhibit a tendency to “forget” samples that were extractable from the pre-trained models. Additionally, our research shows that smaller models are more susceptible to data extraction attacks post-fine-tuning, making them more vulnerable than their larger counterparts. Data carriers and license information are the most likely to be memorized after pre-training. However, while license information is prone to being forgotten during the fine-tuning process, data carriers tend to remain intact, leaving them exposed to potential extraction threats.*

To summarize, the main contributions of this study are:

- The construction of a benchmark for identifying vulnerable samples extractable from models trained on The Stack v2.
- The extension of the study on memorization in code models to include the context

⁴EvalPlus Leaderboard: <https://evalplus.github.io/leaderboard.html>

of the Java programming language.

- An empirical assessment of pre-training data memorization in fine-tuned code models which reveals that fine-tuning reduces the extractability of certain pre-training data from the pre-trained models.
- An empirical assessment of fine-tuning data memorization in code models which shows that (1) code models memorize fine-tuning data; (2) smaller models are more affected by fine-tuning memorization; (3) increasing the duplication rate of samples from the training corpus significantly drives data leakage; (4) similar to pre-training memorization, the input prefix length has a considerable impact on data extractability.

1.2 Thesis Structure

This thesis is organized to present a clear and systematic exploration of the research focused on extracting training data from pre-trained and fine-tuned large language models specialized in code.

Preliminaries: The thesis begins with a chapter that establishes the theoretical foundation necessary for the study. It introduces essential concepts, including language modeling, transformers, and large language models, with a specific focus on their application in software engineering tasks. Additionally, it reviews key data privacy protection laws and code licensing regulations, laying the groundwork for understanding their relevance to the problem of data extraction from pre-trained and fine-tuned models.

Related Work and Problem Definition: The literature review presents a comprehensive examination of recent and relevant research on memorization in large language models, particularly in code models. Topics such as membership inference attacks, targeted and untargeted data extraction attacks, and recent findings on fine-tuning memorization are discussed in detail. These chapters contextualize the current State Of The Art (SOTA), identifying research gaps that this study aims to address.

Methodology: This chapter details the methodologies employed in the study, cover-

ing the processes of dataset construction and model fine-tuning. The data extraction techniques used to retrieve attack samples and execute data extraction attacks are also described. Moreover, a detailed account of the code categorization approach used to analyze different types of code data is provided, ensuring a clear understanding of the experimental setup and methods.

Results: This chapter presents the empirical findings from the experiments, structured around the research questions.

Discussion and Conclusion: The thesis concludes with a critical discussion of the results, offering insights into the nature of the extracted information, the risks posed by such vulnerabilities, and strategies for mitigating these risks. Ethical considerations related to privacy and intellectual property are thoroughly examined, emphasizing the broader implications of deploying LLMs in real-world applications.

1.3 Reproducibility

Reproducibility is a cornerstone of scientific research and is critical for validating and ensuring the reliability of results. In this study, we have adopted a methodological framework designed to be fully accessible for replication and verification. To facilitate this, we provide a comprehensive repository that includes the complete implementation details of the approaches proposed in this thesis with detailed usage instructions. Additionally, we include the datasets utilized for the attack experiments and the fine-tuned models used in the study.

- Replication Package [19].⁵
- Attack Dataset.⁶
- Fine-tuned models.⁷

⁵repo: <https://github.com/fabiosalern/tune-memorization>

⁶attack dataset: https://huggingface.co/datasets/fabiosalern/MEM-TUNE_attack

⁷models: <https://huggingface.co/collections/fabiosalern/mem-tune-models-66f83caa8d215c6331c288ec>

Chapter 2

Preliminaries

This chapter introduces fundamental definitions to enhance the reader’s understanding of the topic. It covers key concepts including language modeling, transformers, and large language models, with a focus on their application in software engineering tasks. Additionally, an overview of data privacy protection laws and code licensing regulations will be presented, establishing a foundation for understanding their relevance in this context.

2.1 Language Modeling

Language is one of humanity’s most powerful tools, enabling the articulation of thoughts and emotions, as well as fostering communication and collaboration across individuals and societies [20]. Despite their advanced computational abilities, machines cannot inherently comprehend and engage in human language. To bridge this gap, Artificial Intelligence (AI) research has focused on developing algorithms capable of reading, writing, and conversing in natural language, closely parroting human interaction [21]. *Natural Language Processing (NLP)* has emerged as a critical subfield within AI, dedicated to advancing this goal.

At the crux of NLP are *language models*, which are assigned with predicting the likelihood of linguistic units (e.g., words, phrases, sentences) within a given context [22]. As illustrated in Figure 2.1, the development of language models has progressed through sev-

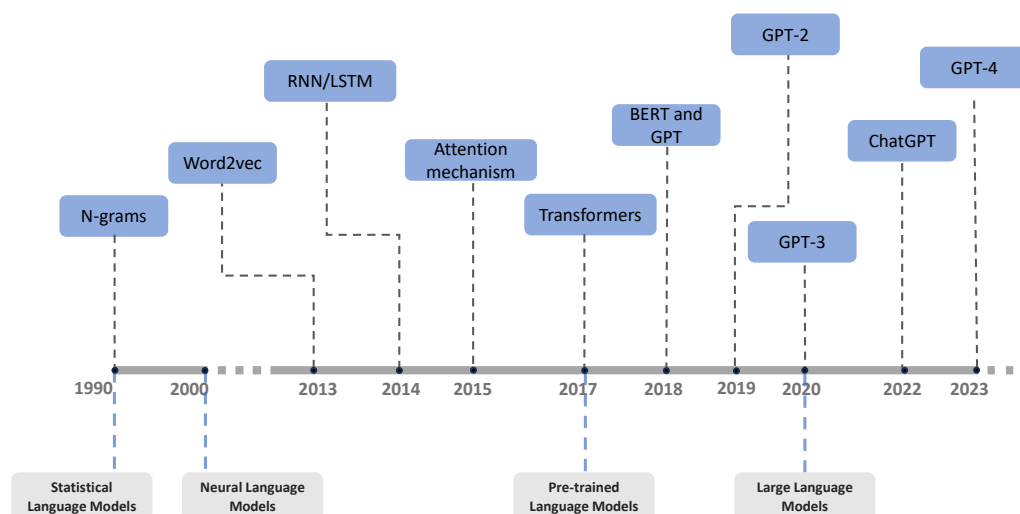


Figure 2.1: Evolution of language models from statistical approaches to large language models [22].

eral key evolutionary stages, beginning with early *Statistical Language Models (SLMs)*, followed by *Neural Language Models (NLMs)*, advancing to *Pre-trained Language Models (PLMs)*, and culminating in the sophisticated *LLMs* we see today. SLMs employ basic probabilistic techniques to model word sequences, whereas NLMs leverage neural networks to capture more intricate patterns and representations within language. PLMs make use of large-scale corpora combined with self-supervised learning to acquire general linguistic knowledge. LLMs build upon the foundations of PLMs by integrating vast amounts of data, computation, and advanced algorithms, resulting in models that are significantly more expressive, adaptive, and capable of handling complex language tasks.

2.2 Statistical Language Models

Statistical language models [23], [24] are grounded in statistical learning methods, which gained prominence in the 1990s as mathematical frameworks designed to address the contextual properties of natural language through probabilistic and statistical approaches. The idea behind SLMs is to predict the next word in a sequence based on prior context,

typically by leveraging the Markov assumption [21]. This assumption asserts that the prediction of the next word depends only on a fixed number of preceding words, not the entire sentence. SLMs that operate on a fixed-length context n are commonly referred to as *n-gram models*, with well-known examples including bigram and trigram models, which use a context size of one and two words, respectively [21].

To illustrate, consider the sentence S : "I am a robot". Here, $P(\omega_i)$ denotes the probability of the i -th word in the sentence: ω_1 as "I", ω_2 as "am", ω_3 as "a", and ω_4 as "robot". The objective is to determine the probability of S appearing in the text, denoted as $P(S)$:

$$P(S) = P(\omega_1, \omega_2, \omega_3, \omega_4) = P(I, am, a, robot). \quad (2.1)$$

This joint probability can be decomposed using conditional probabilities as follows:

$$P(I, am, a, robot) = P(I) \cdot P(am | I) \cdot P(a | I, am) \cdot P(robot | I, am, a). \quad (2.2)$$

In this formula, $P(I)$ represents the probability of the word "I" appearing, and $P(am | I)$ signifies the conditional probability of "am" given that "I" has occurred. By multiplying $P(am | I)$ by $P(I)$, we obtain the probability of the word pair "I am" appearing together, expressed as: $P(I, am) = P(I) \cdot P(am | I)$. This approach can be extended iteratively to calculate the full probability of the sentence. In the case of n-gram models, the process is often simplified by assuming a fixed-length context, such that the model considers only the most recent n words when predicting the next word, reducing computational complexity while retaining contextual relevance.

To be able to compute the probability of the entire sequence, it is essential to calculate the conditional probability for each word in that sequence. This is typically achieved through Maximum Likelihood Estimation (MLE), which allows us to estimate probabilities by replacing them with observed frequencies, assuming the sample size is sufficiently large [23]. The conditional probability of a word w_i , given the preceding words $w_1, w_2, \dots, w_{i-1}$, is calculated as:

$$P(w_i | w_1 w_2 \dots w_{i-1}) = \frac{P(w_1 \cdot w_2 \cdot \dots \cdot w_i)}{P(w_1 \cdot w_2 \cdot \dots \cdot w_{i-1})} = \frac{C(w_1 w_2 \dots w_i)}{C(w_1 w_2 \dots w_{i-1})}. \quad (2.3)$$

Here, $C(\cdot)$ represents the frequency that the subsequence has appeared in the training data [23].

Using this approach, we can determine the likelihood of each word w_i given the sequence of preceding words. The model selects the next word by choosing the one with the highest conditional probability [23].

This method assumes that each word’s probability depends on the preceding $n - 1$ words, aligning with the n-gram model framework. In n-gram models, the conditional probability is computed based on a fixed context size n , where the word sequence $w_1.w_2, \dots, w_{n-1}$ influences the prediction of w_n . To efficiently compute these conditional probabilities, it is necessary to pre-calculate and store the counts $C(X)$, where X is a sequence of length n from the training set [23]. However, the challenge arises due to the exponential growth in the number of possible sequences as the vocabulary size increases. For instance, with a vocabulary of 10,000 unique words, there are $10,000^n$ possible sentences of length n . As n increases, the number of possible sequences becomes computationally expensive to handle, requiring trade-offs between context length and model complexity. This phenomenon is commonly referred to as the curse of dimensionality, where the accurate estimation of high-order n-gram models becomes increasingly difficult. The reason lies in the bare number of transition probabilities that must be estimated, which grows exponentially with the context length n . As a result, high-order n-gram models often suffer from data sparsity and computational inefficiency, necessitating strategies to balance context length and model feasibility [21].

2.3 Neural Language Models

Neural language models [25]–[27] utilize neural networks to predict the probabilities of subsequent words in sequences, effectively addressing the limitations inherent in statistical language models with small values of n . Unlike n-gram models, neural approaches can handle longer contexts and capture more complex dependencies across sequences. These models characterize the probability distribution over word sequences through various neural architectures, such as *Multi-Layer Perceptrons (MLP)* and *Recurrent Neural*

Networks (RNNs) [26], [27]. By leveraging their ability to model non-linear relationships and maintain information over extended sequences, NLMs overcome the curse of dimensionality and data sparsity issues commonly encountered in traditional n-gram-based approaches.

2.3.1 Word Embeddings

A groundbreaking contribution by Bengio *et al.* [25] introduced the concept of *distributed word representations*, fundamentally transforming the way language models are constructed. In their work, they proposed a neural probabilistic language model that addressed key limitations of traditional SLMs, such as the curse of dimensionality and data sparsity. By representing words as continuous-valued vectors in a distributed representation space, the model was able to share statistical strength across similar words, allowing for more efficient learning and generalization, even in high-dimensional input spaces. The key innovation was the introduction of a word prediction function that conditions aggregated context features, represented as distributed word vectors [25]. This model learns both the distributed word representations and the probability function simultaneously, within a unified framework [25]. Specifically, Bengio *et al.* designed a multi-layer neural network to estimate the conditional probability of a word given its context, which consists of a sequence of preceding words. This approach drastically reduces the need for explicit storage of large tables of conditional probabilities, as seen in traditional n-gram models. The model was trained to minimize the negative log-likelihood of the predicted word sequences, and its architecture allowed the context size to scale more effectively, alleviating the limitations associated with fixed n-gram sizes [25]. The authors demonstrated that this neural network-based model could generalize better across unseen word sequences, thanks to the shared parameters in the word embeddings, which captured semantic similarities between words [25].

Building on the foundation established by Bengio *et al.*, Mikolov *et al.* [28] introduced *word2vec*, a more computationally efficient method for learning distributed word representations. Unlike prior models, word2vec employed a simplified, shallow neural network architecture to produce word embeddings that captured both semantic and syntactic

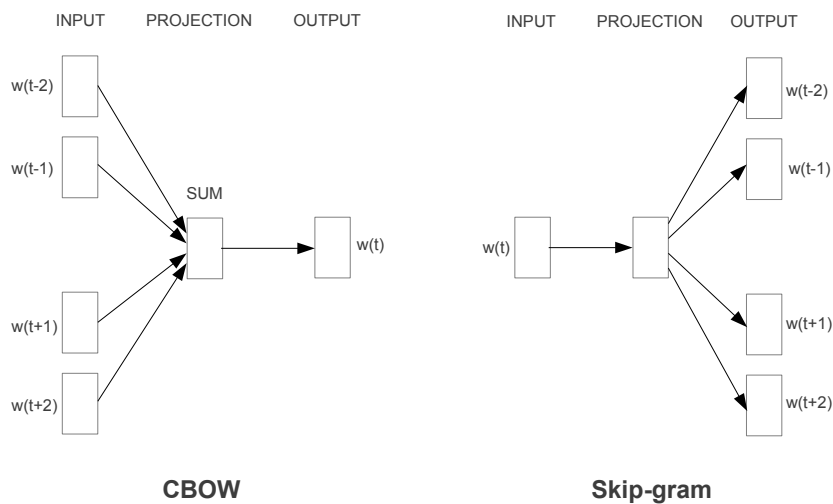


Figure 2.2: The CBOW architecture predicts the current word based on the context and the Skip-gram predicts surrounding words given the current word[28].

relationships between words. Two primary architectures were proposed: the *Continuous Bag-of-Words* (CBOW) model and the *Skip-Gram* model, each optimized for different NLP tasks. The CBOW model predicts a target word based on the surrounding context words, whereas the Skip-Gram model does the reverse predicting context words given a target word. Skip-Gram, in particular, was designed to efficiently handle larger corpora and capture broader semantic relationships by maximizing the likelihood of context words within a defined window around the target word. Both architectures, however, shared the goal of learning dense, low-dimensional vector representations for words that effectively preserve their contextual usage. One of the key innovations in word2vec was the introduction of two techniques to further reduce computational complexity: hierarchical softmax and negative sampling [28]. Hierarchical softmax approximates the full softmax function by using a binary tree, which significantly speeds up the calculation of word probabilities, especially for large vocabularies. Negative sampling, on the other hand, simplifies training by updating only a small number of output neurons for each training instance, instead of the full softmax layer. This approximation method allows the model to focus on distinguishing the correct context words from a few ran-

domly sampled non-matching words (“negative”), making the model much more scalable and efficient. Mikolov *et al.* demonstrated that the embeddings learned by word2vec are remarkably effective in capturing linguistic regularities, such as word analogies. To exemplify, $vector(“King”) - Vector(“Man”) + Vector(“Woman”)$ results in a vector with a close representation to $Vector(“Queen”)$. This approach outperforms traditional distributional approaches in various downstream NLP tasks [28]. The model’s ability to learn high-quality word vectors with relatively simple architecture and fast training times has led to its widespread adoption in NLP applications. However, it’s important to emphasize that word2vec, along with similar early models, operates with static word representations, meaning that these embeddings are context-unaware [21]. Each word is represented by a single vector, regardless of its different meanings or usages in varying contexts. For instance, the word “bank” would have the same embedding whether it refers to a financial institution or the side of a river. This limitation arises because word2vec does not dynamically incorporate the surrounding context when generating the embeddings [21].

2.3.2 Recurrent Neural Networks

Recurrent neural networks are a class of neural networks designed to model sequential data, such as speech and text, by incorporating temporal dependencies via internal memory mechanisms [29]. These mechanisms enable RNNs to store and update information over time, making them suitable for tasks where context and order are crucial, such as language modeling, speech recognition, and time-series prediction [29]. As demonstrated in the work of Mikolov *et al.* [27], RNNs are particularly effective for predicting the probability of a word based on the history of previous words in a sentence, thus capturing both short-term and long-term dependencies within the data. An RNN’s architecture is composed of three primary components: an input layer, a hidden layer, and an output layer. The input layer encodes the input sequence as a series of vectors, typically using word embeddings or other encoding techniques. The hidden layer, which includes one or more recurrent units, processes these input vectors along with the previous hidden state, updating the internal state as it processes each time step in the sequence. Finally,

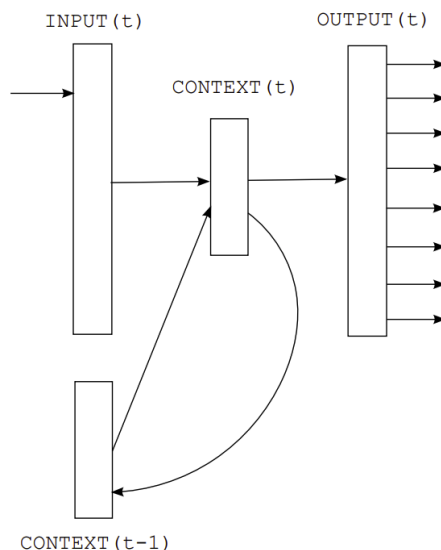


Figure 2.3: Simple recurrent neural network architecture, proposed by Mikolov *et al.* [27].

the output layer generates predictions, often in the form of probability distributions over possible outputs, such as the next word in a sequence [27]. As shown in Figure 2.3, the key innovation in RNNs is the recurrent structure of the hidden layer, which allows the network to maintain a memory of past inputs. However, the standard RNN architecture, which uses activation functions such as $\tanh$ or sigmoid [30], is prone to the vanishing gradient problem during backpropagation [31]. This issue hinders the ability of simple RNNs to learn long-term dependencies, as gradients that propagate through many time steps tend to shrink, effectively preventing the model from learning correlations over long sequences. To address this limitation, advanced recurrent units such as *Long Short-Term Memory (LSTM)* [32] and *Gated Recurrent Units (GRU)* [33] were developed. LSTM incorporates a memory cell along with input, forget, and output gates to regulate the flow of information into and out of the memory. This gated mechanism enables the network to retain relevant information for extended periods, while discarding irrelevant information, thereby alleviating the vanishing gradient problem [32]. GRU, a simplified variant of LSTM, uses fewer gates (i.e., reset and update gates) but retains the ability to capture long-term dependencies effectively [33].

RNNs offer powerful mechanisms for modeling sequential data, but they are not

without substantial drawbacks. Some of the primary challenges associated with RNNs include the following:

- *Vanishing Gradient Problem:* This issue arises when the gradients of the loss function, with respect to model parameters, become exceedingly small or even zero during backpropagation, particularly when propagating through many time steps. As a result, it becomes difficult or impossible for the network to update its weights effectively, making it hard for the model to capture long-term dependencies between input and output elements. RNNs are particularly susceptible to this problem due to the nature of sequential backpropagation, where the gradients can decay exponentially over time. Although LSTMs are designed to mitigate this issue by using gating mechanisms that selectively retain or forget information, they are not entirely immune to the vanishing gradient problem [31].
- *Exploding Gradient Problem:* In contrast to the vanishing gradient issue, the exploding gradient problem occurs when gradients grow excessively large during backpropagation. This can cause the model's parameters to diverge, preventing convergence and leading to unstable training. When gradients become too large, the optimization process fails to update the model weights meaningfully, resulting in a breakdown of the learning process [31].
- *Sequential Computation:* One of the most notable limitations of RNNs and LSTMs is their inherently sequential nature. These models process one element of the input sequence at a time, which slows down both training and inference, particularly for long sequences or high-dimensional data. This sequential processing prevents RNNs and LSTMs from taking full advantage of modern hardware accelerators, such as GPUs and TPUs, which are optimized for parallel computation. As a result, RNN-based models can be computationally inefficient compared to newer architectures like Transformer models, which allow for parallelization and faster training, especially on large-scale data [34].

2.4 Pre-trained Language Models

While static word representations, such as those generated by word2vec, are capable of capturing certain linguistic regularities, such as word analogies, they are inherently limited in representing the dynamic nature of word meaning in context. These static embeddings assign a single vector to each word, regardless of its usage in different contexts, failing to account for polysemy or contextual nuances. To address these limitations, *ELMo* (Embeddings from Language Models) [35] was introduced as one of the early approaches to capture context-dependent word representations. ELMo employs a deep, bi-directional LSTM (biLSTM) architecture that is pre-trained on a large corpus using a language modeling objective. Instead of assigning fixed embeddings, ELMo computes word representations dynamically for each token, conditioned on the entire sentence. This allows the model to adapt word meanings based on their context. The contextualized word representations are then fine-tuned for specific downstream tasks, allowing them to capture both syntactic and semantic subtleties that static embeddings cannot [35].

2.4.1 The Attention Mechanism

Building upon the limitations of sequential models like LSTMs and GRUs, which struggle with capturing long-range dependencies and suffer from bottlenecks in processing efficiency, the *attention mechanism* was introduced as a key innovation [34], [36]. Unlike traditional sequence models that process data step-by-step, attention allows models to dynamically focus on specific parts of the input sequence, regardless of their position in the sequence. This significantly enhances the model’s ability to handle long-range dependencies and enables parallel processing, which is critical for handling longer inputs efficiently. The attention mechanism works by computing a weighted sum of input representations, where the weights signify the relevance or importance of each input element to the task at hand. These weights are not fixed but are learned through a compatibility function that measures the relationship or alignment between different parts of the input and the current processing context. This flexibility allows attention-based models

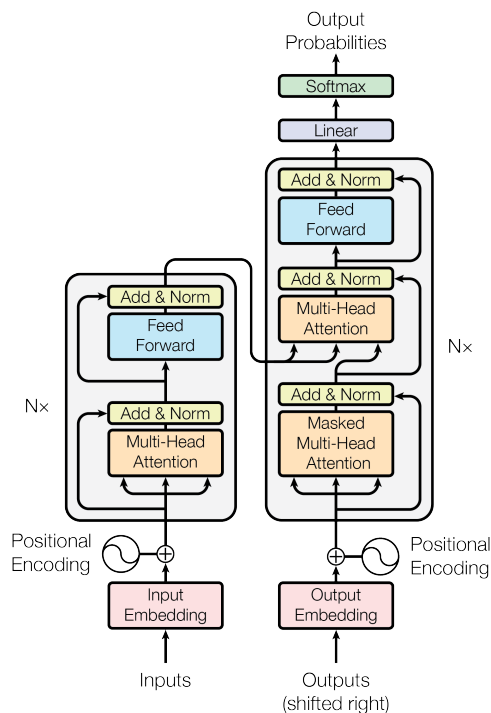


Figure 2.4: The encoder-decoder Transformer architecture introduced by Vaswani *et al.* [34].

to adaptively select relevant information, making them highly effective in tasks such as machine translation, summarization, and other natural language processing applications [21].

2.4.2 The Transformer Architecture

The paper *Attention Is All You Need* [34] proposed a new simple network architecture, the *Transformer*, based solely on the attention mechanism, bypassing recurrence and convolutions. Figure 2.4 illustrates in detail the encoder-decoder Transformer architecture proposed by Vaswani *et al.*, designed for sequence-to-sequence tasks such as machine translation. *Self-attention mechanism* is the core element of the Transformer architecture allowing each token in the input to attend to all other tokens, thereby capturing both local and global context simultaneously. The encoder-decoder Transformer architecture consists of two key components: the *encoder* (left) and the *decoder* (right), both composed of multiple layers, denoted by “ N ”, indicating that they are stacked on top of

Input-Input Layer5

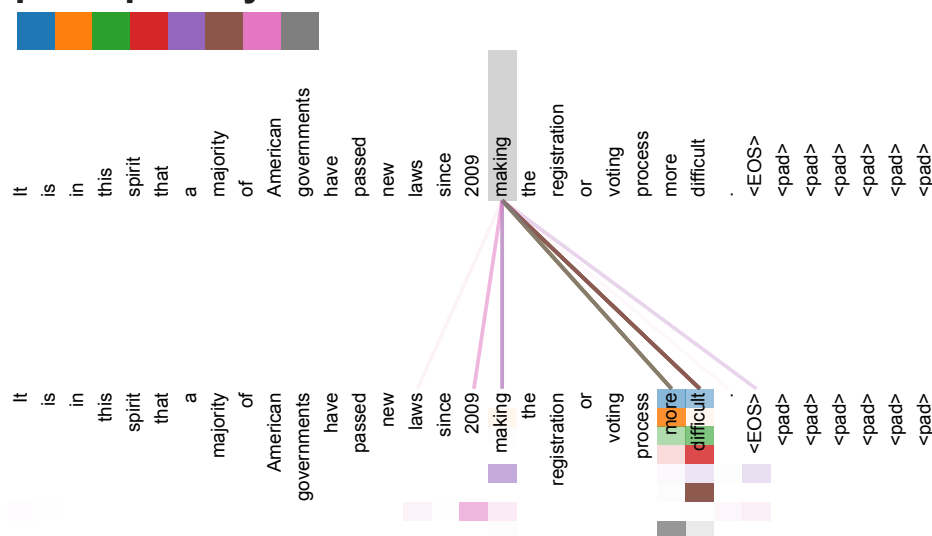


Figure 2.5: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6 [34].

each other. The encoder processes the source sentence by embedding the input tokens and adding positional encodings to retain order information, as self-attention alone lacks inherent sequence awareness. The decoder is responsible for generating the target translation. It processes the output sequence (shifted by one position) and interacts with the encoded source sentence through a series of attention blocks.

Figure 2.5 displays the relationships between tokens in a sequence, where each token’s attention to other tokens is represented by colored lines, indicating the strength or weight of the attention [34]. Attention lines are notably drawn between “making” and itself and words like “more”, and “difficult”. This suggests that, in the model, “making” pays strong attention to these tokens, which aligns with the syntactic and semantic relations in the sentence. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train compared to recurrent models like LSTMs [34].

2.4.3 Pre-training and Fine-tuning Paradigm

Furthermore, based on the highly parallelizable Transformer architecture, *Bidirectional Encoder Representations from Transformers (BERT)* was introduced as a novel approach to pre-train bidirectional language models with an encoder-only architecture [37]. Unlike traditional unidirectional models, BERT’s architecture leverages both left and right context simultaneously during pre-training, enabling it to generate richer and more contextually aware word representations. BERT’s pre-training process involves two novel self-supervised tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). These innovations encode deep bidirectional representations from large-scale, unlabeled text corpora. These pre-trained models, fine-tuned for specific downstream tasks, have demonstrated state-of-the-art performance across a wide range of NLP benchmarks [37]. By capturing deeper semantic nuances, BERT’s representations serve as highly effective, general-purpose semantic features, significantly pushing forward the performance ceiling on various NLP tasks, such as question answering, sentiment analysis, and named entity recognition [37].

This study has inspired and influenced the follow-up work, which sets the “*pre-training and fine-tuning*” learning paradigm. In this approach, PLMs are initially trained on vast amounts of unlabeled text, allowing them to learn essential language properties such as vocabulary, syntax, semantics, and logical reasoning [21]. This stage, known as pre-training, equips the model with a broad understanding of language. Following pre-training, the model is further adapted to specific natural language processing (NLP) tasks such as machine translation, text summarization, or question answering by retraining it on a smaller, task-specific labeled dataset. This subsequent phase, called fine-tuning, tailors the model to perform optimally on the downstream task [21]. Following this paradigm, a great number of studies on PLMs have emerged, leading to the development of diverse architectures such as GPT-2 [38] and BART [39].

2.5 Large Language Models

In recent years, researchers have observed that scaling PLMs, whether by increasing model size or the quantity of training data, often leads to improved performance on various downstream tasks. Numerous studies have focused on exploring the limits of this performance by developing progressively larger PLMs, such as GPT-3 with 175 billion parameters [40] and PaLM with 540 billion parameters [41]. While these scaling efforts have primarily involved increases in model size while maintaining similar architectures and pre-training tasks, the resulting large models exhibit markedly different behaviors compared to their smaller counterparts, such as the 330 million parameters BERT [37] or the 1.5 billion parameters GPT-2 [38]. A fascinating phenomenon is the emergence of novel capabilities referred to as “*emergent abilities*” absent in smaller models. For instance, GPT-3 has demonstrated the ability to perform few-shot learning through in-context learning, a capability that GPT-2 lacks [40]. These emergent abilities enable language models to solve a broader range of complex tasks with minimal task-specific training or examples. This has led to the introduction of the term *large language models* to describe these high-capacity models, which have gained increasing attention from the research community due to their potential for significant advancements in natural language processing and other AI-driven disciplines [42].

The technological development of language models has significantly expanded their task-solving capabilities. Initially, SLMs were employed for specific tasks (e.g., speech task or retrieval), where predicted probabilities improved the performance of task-specific methods. NLMs then pivoted the focus to task-agnostic representations (e.g., embeddings), reducing the necessity for manual feature engineering. Pre-trained language models further extended this by learning context-aware representations optimized for various downstream tasks. For the latest generation of language models, LLMs are enhanced by exploring the scaling effect on model capacity, which can be considered as general-purpose task solvers. Across the board, in the evolution process, the range of tasks addressable by language models has widened, and their performance has significantly improved [21].

The work by Kaplan *et al.* provides a deeper understanding of how scaling laws affect

the performance of PLMs [43]. Their findings suggest that performance improvements from scaling are predictable based on a set of scaling laws, which relate model size, dataset size, and computational resources to the overall performance on a variety of benchmarks. This discovery has contributed to the widespread interest in scaling PLMs, as researchers aim to optimize resource allocation and design strategies for training larger models more efficiently. Furthermore, Kaplan *et al.* suggests that there is a critical threshold at which these emergent abilities become pronounced, raising questions about the nature of intelligence and generalization in artificial models. As a result, the ongoing research into LLMs is now focused on identifying the conditions that lead to emergent abilities, optimizing scaling strategies, and addressing the growing challenges associated with model interpretability, fairness, and resource efficiency [43].

2.5.1 Model Training

Regardless of the concrete model architecture, current LLM training typically encompasses three main stages: pre-training, supervised fine-tuning, and reinforcement learning from human feedback [21].

Pre-training constitutes the foundational phase in the training pipeline of LLMs, enabling them to acquire fundamental language understanding capabilities applicable to a wide range of language-related tasks. During pre-training, LLMs are trained on massive amounts of unlabeled text data in a self-supervised manner [44]. Common pre-training approaches include autoregressive language modeling, where the model predicts the next token given the previous context [40], and masked language modeling, where certain tokens in a sequence are masked and the model learns to predict these masked tokens based on the surrounding context [37].

Early language models, such as BERT [37], trained using self-supervision (as explained in Section 2.4.3), were not able to perform specific tasks without additional adaptation. To make these foundation models useful for particular applications, they needed to be fine-tuned on specific tasks using labeled data. This is a process known as supervised fine-tuning (SFT). While more recent LLMs can perform a variety of tasks without explicit fine-tuning, they can still benefit from task- or data-specific fine-tuning.

For example, OpenAI informs that fine-tuning smaller models like GPT-3.5 Turbo with task-specific data can conduct performance improvements that, in some cases, overtake larger models like GPT-4 [45]. The core concept of supervised fine-tuning involves adjusting the model in a supervised manner based on large-scale pre-training, thereby enhancing its capability to better adapt to the specific requirements of the target task [21].

Alignment is the process of guiding AI systems towards human goals, preferences, and principles. LLMs, pre-trained for word prediction, often exhibit unintended behaviors. For example, they might generate content that is toxic, harmful, misleading, and biased [46]. Reinforcement Learning from Human Feedback (RLHF) is a popular alignment method for large language models which consists of optimizing the fine-tuned model to produce generations that maximize the score assigned by a reward model typically learned directly from user preference data [47].

2.5.2 Model Generation

Decoding in LLMs is the process by which these models generate text based on an input prompt. Upon receiving an input, the tokenizer converts the text into corresponding token IDs, which are then used by the LLM to predict the next most likely token (or sequence of tokens). These predictions result in *logits*, which are converted into probabilities using a softmax function [30]. Several decoding strategies have been developed to guide this token-generation process.

Greedy Search: The greedy search method selects the most probable token at each step of the generation process, disregarding all other possibilities. While this approach is computationally efficient and straightforward, it often leads to a loss in coherence over longer sequences due to its short-sighted nature, as it fails to consider how earlier decisions influence the overall sequence quality. Thus, although greedy search can rapidly produce results, it may overlook alternative sequences that contain slightly less probable tokens but exhibit superior overall consistency [48].

Beam Search: In contrast, beam search maintains multiple potential token sequences simultaneously, considering the top N most probable tokens at each step, where N repre-

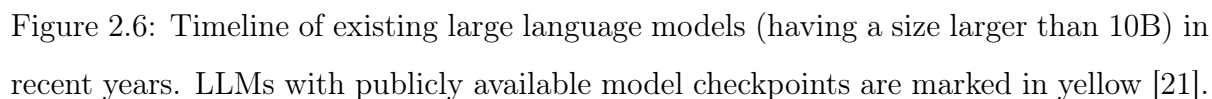
sents the beam width. The process continues until the sequence reaches a predetermined maximum length or an end-of-sequence token is generated. The sequence with the highest cumulative probability is selected as the final output. However, beam search’s ability to evaluate numerous potential sequences introduces a higher computational cost. For example, a beam size of 2 with a sequence length of 5 necessitates tracking $2^5 = 32$ potential sequences, leading to a trade-off between computational efficiency and accuracy [48].

Top- k sampling: The top- k sampling approach introduces randomness by selecting a token from the top k most probable tokens according to the model’s probability distribution. For instance, consider a scenario with six tokens (A, B, C, D, E, F) where $P(A) = 30\%$, $P(B) = 20\%$, and $P(C) = P(D) = P(E) = P(F) = 12.5\%$. If $k = 2$, tokens C through F are disregarded, and tokens A and B are sampled based on their probabilities, with A selected 60% of the time and B selected 40% of the time. This method balances probability with randomness, enhancing diversity in the generated text [48].

Top- p sampling: The top- p sampling method is a refinement of the Top- k approach by selecting tokens based on a cumulative probability threshold p . Instead of limiting the selection to the top- k tokens, top- p (nucleus) sampling includes tokens in descending order of probability until the cumulative probability exceeds p . This dynamic selection process leads to a more flexible token set than top- k sampling, enabling the model to adapt to cases where high-probability tokens are limited, promoting diversity in output while maintaining relevance [48].

2.5.3 State of The Art Large Language Models

As illustrated in Figure 2.6, since 2019, the field of LLMs has grown exponentially, leading to the emergence of numerous projects in both open-source and closed-source domains. Open-source LLMs are characterized by publicly accessible model weights and, in some cases, accessible training data [21]. This transparency enables researchers and developers to examine, modify, and build upon existing models without significant barriers.



Generative Pre-trained Transformers (GPT) is a family of decoder-only Transformer-based language models developed by OpenAI [44]. While early GPT models like GPT-1 and GPT-2 were open-sourced, more recent models such as GPT-3 and GPT-4 have not been publicly released. GPT-3 [40] is a pre-trained autoregressive language model with 175 billion parameters. It is widely considered the first large language model not only because of its unprecedented size but also due to its emergent abilities not observed in previous smaller PLMs. To enhance the alignment of LLMs with human instructions,

24

OpenAI introduced InstructGPT3 [49]. This model is fine-tuned with human feedback to better follow user intent across a wide range of tasks. The fine-tuning process involves collecting a dataset of prompts and desired outputs written by human labelers, as well as prompts submitted through the OpenAI API. GPT-3 is then fine-tuned on this dataset. Additionally, human-ranked model outputs are used to further refine the model using RLHF [49]. A substantial milestone in LLM development is the launch of ChatGPT² on November 30, 2022. ChatGPT is a chatbot that allows users to engage in conversations to accomplish a wide array of tasks, including question-answering, information-seeking, and text summarization. Powered by GPT-3.5 (and later GPT-4), ChatGPT is trained to follow instructions in prompts and provide detailed responses. GPT-4 [45], launched in March 2023, is the latest and most powerful model in the GPT family. It is a multimodal LLM capable of processing both image and text inputs to produce text outputs. While it still falls short of human capabilities in some real-world scenarios, GPT-4 demonstrates performance that reaches the human level on various professional and academic benchmarks, including passing a simulated bar exam with a score in the top 10% of the participants [45].

LLaMA is a collection of foundational language models released by Meta [50]. Unlike GPT models, LLaMA models are open-source; their model weights are made available to the research community under a non-commercial license. This openness has led to rapid growth in the LLaMA family, as numerous research groups utilize these models to develop enhanced open-source LLMs that can compete with closed-source alternatives or create task-specific applications. The initial set of LLaMA models [50], released in February 2023, ranges from 7 billion to 65 billion parameters. These models are pre-trained on trillions of tokens collected from publicly available datasets. LLaMA embraces the transformer architecture used for GPT-3 with a few minor architectural modifications [50]. In July 2023, Meta, in partnership with Microsoft, released the LLaMA-2 collection [10], which includes both foundational language models and chat models fine-tuned for dialogue, known as LLaMA-2 Chat. The LLaMA-2 Chat models reportedly outperform other open-source models on numerous public benchmarks [10]. In April 2024, Meta

²Introducing ChatGPT: <https://openai.com/index/chatgpt/>

revealed LLaMA-3 [51], their latest open-source large language model, with significant improvements over the previous generation. LLaMA-3 has been presented in two versions with 8B and 70B parameters, enhanced reasoning, coding abilities, and multilingual support. It aims to foster community-driven innovation with an open approach. Meta emphasizes responsible AI deployment, offering tools like LLaMA Guard 2 and Code Shield to ensure safety and reliability [51].

2.5.4 Drawbacks and Risks of Large Language Models

Despite being one of the most transformative advancements in the field of artificial intelligence, LLMs exhibit several limitations that present significant challenges for their deployment in critical applications.

Privacy concerns: LLMs are increasingly deployed across various sectors, yet they raise substantial privacy concerns. One prominent issue is the risk of data leakage, where LLMs may inadvertently memorize and reproduce sensitive or personal information from their training datasets, potentially leading to privacy breaches. Carlini *et al.*, has demonstrated that LLMs can leak sensitive information when subjected to targeted attacks, highlighting the vulnerabilities inherent in these systems. This risk is particularly pronounced in sensitive domains such as healthcare and finance, where the exposure of private data could lead to severe consequences. Moreover, the training datasets used for LLMs are often not fully anonymized, meaning personally identifiable information and other sensitive data could be embedded within the model [3]. This consistently increases the risk of violating privacy regulations, such as the General Data Protection Regulation (GDPR) [52]. These concerns underscore the urgent need for robust privacy safeguards and regulatory frameworks to prevent the misuse of LLMs and to protect sensitive information from malicious exploitation.

Bias and discrimination: LLMs are widely employed for decision-making across various domains, affecting individuals and society. However, LLMs are prone to exhibiting various forms of bias, originating from the input training data [53]. These biases may result in discriminatory or bigoted responses that can reinforce harmful stereotypes. The sources of discrimination include biased training data, and human-interaction bias, lead-

ing to tilted or inaccurate outputs that may perpetuate existing inequalities. Addressing bias in LLMs requires careful curation of datasets and the introduction of bias mitigation techniques at multiple stages of model development [53].

Reliability and hallucinations: LLMs may generate outputs that diverge from the provided context, user input, or factual knowledge, a phenomenon referred to as “hallucination” [54]. These hallucinations arise because LLMs rely heavily on patterns in the data rather than real-world understanding, and their probabilistic nature may produce plausible-sounding yet incorrect information. This undermines their reliability in real-world scenarios, emphasizing the vital necessity for precision, particularly in critical domains [54]. As a primary way to avoid these problems, well-aligned LLMs can be developed by including humans in the training loop [54]. Using RLHF and including safety-related cues in this process, GPT-4 has shown relevant improvements [45]. However, RLHF relies heavily on high-quality human feedback data from professional annotators, which is costly and time-consuming [22].

Intellectual property concerns: LLMs possess the faculty to create content mimicking human-generated material, potentially violating users’ Intellectual Property (IP) rights [22]. These models rely on training and optimizing using users’ textual data, which may encompass personal information, proprietary knowledge, patented technology, and more [55]. Notably, AI-generated code exemplifies this issue. Code-generation models trained on open-source repositories can produce programs resembling existing ones, potentially disregarding relevant licenses [55]. Katzy *et al.* found that datasets used to train LLMs for code contain non-permissively licensed code, even those that claim otherwise [56]. Concerns about intellectual property extend beyond code to include LLM-generated images, texts, and other creative works. Notably, legal actions have been taken against companies like OpenAI, including lawsuits in July 2023 alleging the use of copyrighted books to train models without proper authorization, and a case filed by The New York Times in December 2023 for scraping articles without consent ³. These cases underscore the need for enhanced regulations and legal frameworks governing IP rights regarding

³NY Times sues OpenAI, Microsoft for infringing copyrighted works: <https://www.reuters.com/legal/transactional/ny-times-sues-openai-microsoft-infringing-copyrighted-work-2023-12-27/>

LLMs.

While LLMs hold extraordinary potential for a wide range of applications, their limitations must be addressed to ensure their responsible and ethical deployment, particularly in sensitive and high-stakes domains. An effort to mitigate these challenges through both technical advancements and regulatory measures is essential for exploiting the benefits of these models while minimizing their associated risks.

2.6 Large Language Models for Software Engineering

Software Engineering (SE), is a discipline focused on the development, implementation, and maintenance of software systems [57]. Recently, LLMs have increasingly been conceived specifically for the code intelligence domain. The integration of LLMs into SE arises from a novel approach, where a wide range of traditional SE challenges are reinvented as tasks involving data, code, and textual analysis. This shift has opened new avenues for automating complex tasks, improving software development efficiency, and addressing challenges related to code comprehension, debugging, and generation [58].

Codex [2], an LLM with 12 billion parameters, has demonstrated important capabilities in code generation, successfully solving 72.31% of complex Python programming challenges posed by humans [2]. This pioneering achievement has showcased the potential of LLMs in code-processing tasks. The success of Codex has led to the emergence of commercial products such as GitHub Copilot ⁴ and open-source, multi-billion-parameter code models like StarCoder [59] and Code LLaMA [60].

2.6.1 Downstream Tasks in Code Language Models

As extensive open-source datasets become increasingly available, *Code Language Models (CLMs)* trained on billions of lines of code are being integrated into various stages of the software development life cycle, including requirements engineering, development, testing, deployment, and operations [58]. Table 2.1 summarizes the SE tasks addressed by

⁴GitHub Copilot: <https://github.com/features/copilot>

Table 2.1: Summary of LLMs with different architectures used in SE tasks [1].

Model architecture	Type	Example of SE tasks
Encoder-only	Understanding	Code Understanding Bug localization Vulnerability detection
Decoder-only	Generation	Code generation Code completion Test case generation
Encoder-Decoder	Understanding and Generation	Code summarization Code translation Program repair

LLMs, categorized by their model architecture: encoder-only, decoder-only, and encoder-decoder [1].

Encoder-only LLMs are neural network architectures that utilize only the encoder component of the Transformer model. The encoder’s role is to process and encode the input sentence into a hidden representation, aiming to capture the relationships between words and the overall context of the sentence. Prominent examples of encoder-only LLMs include BERT [37] and its variants. Encoder-only LLMs have shown efficacy in tasks requiring a nuanced understanding of entire sentences and in the SE domain of entire code snippets. Examples of applications include code review, bug report understanding, and code entity recognition. In the SE domain, models like CodeBERT [61], and GraphCodeBERT [62] have been widely employed [1].

Decoder-only LLMs exclusively utilize the decoder module to generate the target output text, following a specific training paradigm that underscores sequential token prediction. In this architecture, the model begins with an initial state and iteratively predicts subsequent tokens, gradually constructing the output text. This approach counts heavily on the model’s ability to comprehend and foresee language structure, syntax, and context. Central implementations of decoder-only LLMs include the GPT series models, with Codex [2] serving as fine-tuned versions tailored for specific SE tasks. Decoder-

only LLMs are particularly well-suited for various text generation tasks, such as code generation and code completion. These models often perform downstream tasks using only a few examples or simple instructions, without additional prediction heads or extensive fine-tuning. This capability makes them valuable tools in the SE domain. Several open-source code models, such as Pythia [63], StarCoder [59], Code LLaMa [60], and SantaCoder [64], also adopt the decoder-only architecture [1].

Encoder-decoder LLMs combine both encoder and decoder components to process and then generate text effectively. The encoder consumes the input sequence and encodes it into a hidden representation that captures the underlying syntactic and semantic features. This hidden intermediate representation bridges the gap between diverse input and output formats. The decoder then utilizes this representation to generate the target output text, translating the abstract encoding into concrete and contextually pertinent expressions. Prominent examples of encoder-decoder LLMs are T5 [65], its variants, and BART [39]. The encoder-decoder architecture offers flexible training strategies and excels in handling complex tasks such as summarization, translation, and question-answering. In the SE domain, this architecture has been successfully applied to tasks like code summarization. The encoder’s capacity to comprehend and represent both the structural and semantic aspects of code is crucial, enabling the decoder to translate this understanding into concise, human-readable summaries. Models such as CodeT5 [66], its successor CodeT5+ [67], and AlphaCode [68] embody this architecture showcasing adaptability to various SE tasks [1].

2.6.2 Data in Code Language Models

Data is an indispensable and critical factor in training LLMs, as it defines the models’ generalization capability, effectiveness, and overall performance [69]. According to Hou *et al.*, data sources for training CLMs can be divided into four categories [1].

- *Open-source datasets*: It refers to publicly accessible collections of data often spread through open-source platforms or repositories [2], [70], [71].
- *Collected datasets*: It refers to data scraped from a multitude of sources, includ-

ing major websites like GitHub ⁵ and forums like Stack Overflow ⁶. GitHub is a platform for hosting and collaborating on software development projects, providing access to a vast array of code repositories. Stack Overflow is a question-and-answer site where professional and enthusiast programmers share knowledge, offering a rich source of programming-related discussions and solutions [8], [59].

- *Constructed datasets*: It includes specialized datasets created by researchers who modify or augment collected datasets to better align with specific objectives. These modifications can be carried out through manual or semi-automatic methods and may include the generation of domain-specific test sets, annotated datasets, or synthetic data. For instance, researchers might take a collected dataset of code snippets and manually annotate them with bug types to create a constructed dataset for classifying code issues [72], [73].
- *Industrial datasets*: These are obtained from commercial or industrial entities and often contain proprietary business data, user behavior logs, and other sensitive information. These datasets are particularly valuable, but their acquisition is often complicated by issues related to business confidentiality and data privacy [74].

In recent years, many large-scale code pre-training datasets have been introduced Table 2.2 summarizes the most relevant ones [58]. CodeSearchNet [75] consists of millions of code functions written in six programming languages (Python, Java, JavaScript, Go, PHP, and Ruby) extracted from GitHub. Its primary purpose is to facilitate semantic code search, where a model retrieves relevant code based on a natural language query. The dataset includes over 6 million functions, with 2 million of them containing accompanying documentation, such as docstrings. CodeSearchNet has been used to train models for code retrieval, code summarization, and code documentation generation. The Pile [76] includes diverse data from 22 sources, such as academic papers, web pages, and books, intending to provide a comprehensive and diverse training set. It is designed to cover multiple domains, including scientific research, legal documents, and programming

⁵<https://github.com/>

⁶<https://stackoverflow.com/>

data, making it useful for a wide range of natural language processing tasks. CodeParrot [77] consists of Python code extracted from open-source projects, it is used to train models on tasks like code generation and completion. The dataset focuses on Python code snippets and functions from public GitHub repositories. It is intended to enhance model capabilities in generating, understanding, and predicting Python code structures. CodeXGLUE [2] is a collection of benchmark datasets and tasks for code-related machine learning. It includes datasets for various code intelligence tasks, such as code completion, code summarization, and code translation between programming languages.

Table 2.2: Main CLMs datasets comparing size, number of files, and number of programming languages (# PL) [58].

Dataset	Size (GB)	Files (M)	# PL
CodeSearchNet [75]	20	6.5	6
CodeXGLUE [2]	20	4.5	10
The Pile [76]	95	19	-
CodeParrot [77]	1K	115	1
The Stack [59]	3.13K	317	30
The Stack v2 [8]	67.5K	3K	619

To effectively train and apply LLMs in SE tasks, it is essential to preprocess raw datasets to obtain clean and suitable data for model training. This preprocessing involves operations such as data cleaning, malware removal, and deduplication. Ensuring data consistency and quality may require different processing methods tailored to specific data types, models, and objectives thereby enhancing the performance and effectiveness of CLMs in these tasks. The subsequent generation of The Stack [59], The Stack v2 [8] is a high-quality, ethically sourced dataset designed for pre-training CLMs such as StarCoder2 [8]. It contains a broad range of open-source code, documentation, notebooks, and related materials. Lozhkov *et al.* provides a detailed description of the data preparation pipeline employed in the code data share for this dataset, which is visually summarized in Figure 2.7. The initial phase of data preparation involves extracting

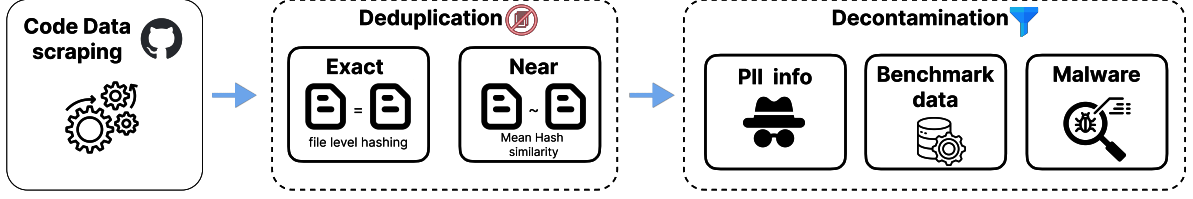


Figure 2.7: Pre-processing pipeline implemented by Lozhkov *et al.* for The Stack v2 [8].

and filtering code from millions of GitHub repositories. This process begins by applying several layers of filters to the most recent version of each repository scraped, focusing exclusively on the main branch to ensure that only the latest, stable code is retained for analysis. Following this, a de-duplication step is performed to eliminate near-duplicate files. MinHashes are computed to quantify the similarity between file contents, and Locality Sensitive Hashing (LSH) is employed to cluster files with similar MinHash fingerprints into groups, marking them as duplicates [8]. Subsequently, the data undergoes a process of personal data sanitization, where personally identifiable information such as names, emails, passwords, IP addresses, and usernames are removed. This is accomplished using the StarPII model [8]. To preserve conversational flow in code comments, pseudonyms like “username_1” are substituted for actual usernames, with only active participants’ identifiers being anonymized. Next, a decontamination procedure is conducted to ensure the training data does not overlap with test sets used in major benchmarks, thereby preventing any artificial inflation of StarCoder’s performance on evaluation metrics. Finally, a malware removal step is performed to safeguard the dataset against potentially harmful code. For this purpose, ClamAV version 1.2 is utilized to scan the training set and identify any instances of malware [8].

2.6.3 Evaluating Code Language Models

Evaluating the performance of CLMs is a crucial aspect of their development and deployment since these models are increasingly integrated into software development workflows. GitHub Copilot and similar tools can generate, refactor, and optimize code, accelerating development processes. However, without proper evaluation, the quality and reliability of the code they produce can vary widely, raising concerns about correctness, security,

and maintainability. One critical area of this inspection is code quality, which includes verifying the accuracy, efficiency, and security of the generated code. Faulty or inefficient code can lead to security vulnerabilities or performance issues. Therefore, assessing how well the code meets functional requirements and adheres to best practices, such as those regarding readability and maintainability is key for its integration into production environments [1]. Another noteworthy aspect is contextual accuracy. LLMs trained on vast datasets must be tested to ensure they can generate code that fits within the context of specific projects, including real-world dependencies and evolving codebases. Additionally, LLMs must be evaluated for their ability to handle complexity and avoid producing incorrect or overly simplistic solutions, which can happen even when models output code with high confidence. This is why rigorous testing against real-world programming challenges is necessary to avoid downstream errors [1].

Benchmarking against existing datasets and using baselines are common practices to evaluate the effectiveness of LLMs and this also applies to CLMs. However, given the diversity of SE tasks, a single evaluation metric may not capture the model’s performance comprehensively. Thus, researchers often employ a range of evaluation metrics tailored to specific problem types. Table 2.3 gives an overview of the most common evaluation metrics for different types of tasks [1]. For classification tasks, the most commonly used metrics are Precision, Recall, and F1-score [1]. For recommendation tasks, MRR (Mean Reciprocal Rank) is the most frequent metric [1]. This metric focuses on the position of the first relevant result in a list, MRR is the average of these reciprocal ranks across multiple queries, providing an overall assessment of how well a system retrieves relevant documents at top positions. It is typically employed to measure the effectiveness of recommendation systems for code completion [1]. Generative models for code are predominantly benchmarked by matching samples against a reference solution, where the match can be exact (*Exact Match (EM)*) or fuzzy (e.g. *Bilingual Evaluation Understudy (BLEU)* [78]). However, recent work has surfaced deficiencies in match-based metrics for code noticing problems in capturing semantic features specific to code [61]. Kulal *et al.* evaluate functional correctness using the *pass@k* metric, where k code samples are generated per problem, a problem is considered solved if any sample passes

Table 2.3: Evaluation metrics for different types of tasks [1].

Problem Type	Metric	
Regression	MAE (Mean Absolute Error)	
Classification	Precision, Recall, Accuracy	F1-score, AUC
	FPR / FNR (False Positive/ Negative Rate)	ROC (Receiver Operating Characteristic)
Recommendation	MAP/MAP@k	MRR (Mean Reciprocal Rank)
	Recall@k, Precision@k, F-score@k	Accuracy
Generation	BLEU/BLEU-4, CodeBLEU	ROUGE/ROUGE-L, METEOR
	Pass@k	Accuracy@k, Precision, Recall, F1-score
	EM (Exact Match)	ED (Edit Distance), ES (Edit Similarity)
	PP (perplexity)	CodeBERTScore, CrystalBLEU

the unit tests, and the total fraction of problems solved is reported [79].

Benchmarks play a crucial role in evaluating the performance of CLMs by providing standardized coding tasks that measure various capabilities such as code understanding, generation, and reasoning. Typically, a benchmark comprises datasets and evaluation criteria designed to assess how well a model can perform a specific set of tasks. These standardized tests allow researchers and developers to objectively compare models, highlight strengths and weaknesses, and set targets for future improvements. One notable example is the *HumanEval* dataset,⁷ released by OpenAI includes 164 Python programming problems with a function signature, docstring, body, and several unit tests. Austin *et al.* introduce MBPP (Mostly Basic Programming Problems), it contains 974 short Python functions designed to be solved by entry-level programmers, text descriptions of those programs, and test cases to check for functional correctness [71]. These two benchmarks have emerged as some of the most influential benchmarks for code models in recent years. However, as noted by Liu *et al.*, existing benchmarks often rely heavily on manually constructed test cases, which can be insufficient for comprehensively evaluating CLMs. Crafting high-quality test cases is a labor-intensive process, and many existing benchmarks fail to cover all potential edge cases, leading to false confidence in

⁷https://huggingface.co/datasets/openai/openai_humaneval

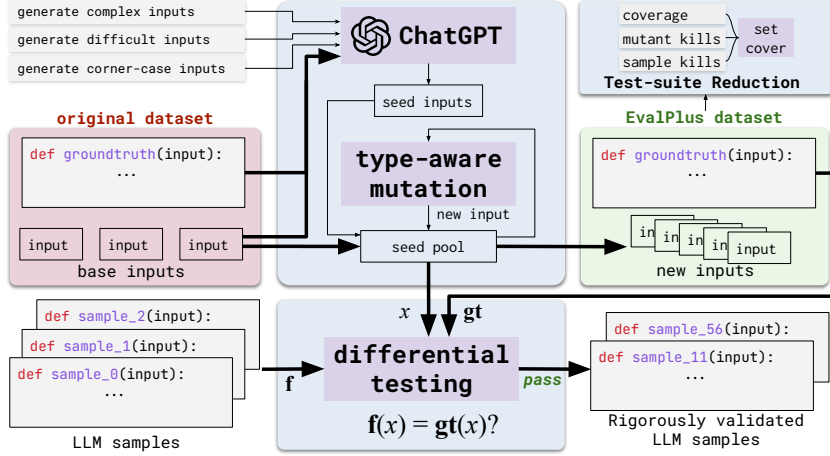


Figure 2.8: Overview of EvalPlus automatic test input generation engine [18].

the performance of LLM-generated code [18]. Particularly, common limitations include insufficient testing, many programming benchmarks providing fewer than 10 tests per coding problem on average, and ambiguous problem descriptions, which often lack clarity about expected program behavior. To address these limitations, Liu *et al.* developed *EvalPlus*, an evaluation framework aimed at more precisely assessing the functional correctness of LLM-generated code. The core innovation of EvalPlus is an automatic test input generation engine that enhances existing benchmarks by generating a diverse set of test inputs to evaluate the code’s functionality thoroughly. As can also be seen from Figure 2.8, EvalPlus begins by using ChatGPT to generate a set of high-quality seed inputs that are designed to test challenging corner cases while adhering to the valid input structure. Building upon these seed inputs, EvalPlus applies type-aware mutation techniques to generate a large number of additional test inputs efficiently. These newly generated inputs are then used for differential testing, comparing the outputs of the LLM-generated code against a ground-truth implementation. This approach enables a more rigorous and reliable evaluation of LLM-generated code [18].

2.7 Personal Data Compliance in the European Union

Rapid advancements in *Information Communication Technologies (ICTs)*, particularly in areas such as LLMs, introduce consequential new safety, privacy, and security challenges.

These technologies, which rely heavily on vast amounts of data for their development and operation, are at the heart of what is increasingly known as the “data economy” a system where data is treated as a critical economic asset. In this data-centric economy, personal information is continually collected, processed, and exchanged, making the protection of privacy and data security more complex than ever before. To address these technological advancements, robust regulations and policies are essential. These regulations must not only safeguard individual rights but also keep pace with the dynamic nature of technology to ensure that ethical standards are maintained in data handling and usage. In Europe, the main example of such regulation is the *GDPR* [52], a comprehensive law designed to protect the privacy and rights of individuals in the European Union (EU) and the European Economic Area (EEA). Passed in 2016 and fully enforceable by 2018, the GDPR replaced the earlier Data Protection Directive 95/46/EC and established a modern framework for managing personal data in an increasingly globalized digital environment. The regulation mandates strict guidelines for how organizations collect, process, store, and transfer personal data, both within the EU and EEA and across international borders. Importantly, it applies to any entity that handles the data of EU or EEA residents, regardless of the organization’s location, thus extending its reach globally. As ICTs like LLMs continue to evolve, regulations like the GDPR will need ongoing updates and refinements to ensure that they remain relevant and effective in safeguarding from emerging technological advancements.

Foundational Definitions (Art. 4): Article 4 of the GDPR provides the foundational definitions of key terms used throughout the regulation [52].

- **Art. 4:** (1) ‘personal data’ means any information relating to an identified or identifiable natural person (‘data subject’); an identifiable natural person is one who can be identified, directly or indirectly, in particular by reference to an identifier such as a name, an identification number, location data, an online identifier or to one or more factors specific to the physical, physiological, genetic, mental, economic, cultural or social identity of that natural person;
- **Art. 4:** (2) ‘processing’ means any operation or set of operations which is per-

formed on personal data or on sets of personal data, whether or not by automated means, such as collection, recording, organization, structuring, storage, adaptation or alteration, retrieval, consultation, use, disclosure by transmission, dissemination or otherwise making available, alignment or combination, restriction, erasure or destruction;

Article 4(1) of the GDPR defines the term personal data as any information that relates to an identified or identifiable natural person, defined as the “*data subject*”. A person is considered identifiable if they can be identified, directly or indirectly, using various identifiers, such as a name, identification number, location data, or online identifiers, as well as through factors that relate to their physical, physiological, genetic, mental, economic, cultural, or social identity. This definition establishes a broad understanding of personal data and emphasizes the importance of considering not just direct identifiers (e.g., names) but also indirect identifiers (e.g., online activity) that, when combined, can reveal an individual’s identity. In this context, PII is a closely related concept often used in privacy discussions, particularly outside the EU. Article 4(2) of the GDPR defines processing as any operation or set of operations carried out on personal data. This includes collecting, recording, organizing, structuring, storing, adapting, retrieving, using, transmitting, disseminating, aligning, combining, restricting, erasing, or destroying data.

The Rights of The Data Subject (Art. 12-23): The rights of the data subject under GDPR (Articles 12-23) [52] grant individuals significant control over their personal data. Key rights include the right to transparent information (Art. 12), ensuring that data subjects receive clear communication about data processing, and the right of access (Art. 15), allowing individuals to know what data is being collected and how it’s used. Data subjects have the right to rectification (Art. 16) to correct inaccurate data, and the right to erasure (Art. 17), often called the “right to be forgotten”, allowing data deletion under specific conditions. The right to restrict processing (Art. 18) gives individuals the ability to limit how their data is used in certain situations, while the right to data portability (Art. 20) allows the transfer of their data between services. Furthermore, individuals can object to processing (Art. 21), especially in cases of direct marketing or where

processing is based on legitimate interests. Lastly, the regulation protects automated decision-making and profiling (Art. 22), ensuring individuals are not subjected to solely automated decisions without explicit consent or contractual necessity while limiting these rights under certain circumstances (Art. 23). These requirements safeguard individuals and collectively strengthen data protection and control.

Controller and Processor (Art. 24-43): In the GDPR framework, the Controller and Processor play crucial roles in ensuring data protection compliance [52]. The Controller is the entity that determines the purposes and means of processing personal data and bears primary responsibility for implementing appropriate technical and organizational measures to ensure compliance with GDPR principles such as transparency, security, and lawfulness (Art. 24-31). This includes obligations such as ensuring data protection by design and by default, maintaining records of processing activities, and reporting data breaches to authorities. The Processor, on the other hand, processes personal data on behalf of the Controller and must strictly follow the Controller’s instructions while implementing necessary security measures (Art. 28-43). Processors are also required to notify Controllers of any data breaches and must ensure any sub-processors are GDPR-compliant. Both Controllers and Processors must cooperate with supervisory authorities, designate Data Protection Officers (DPOs) under certain conditions, and, in some cases, conduct Data Protection Impact Assessments (DPIAs) to evaluate high-risk processing activities. Together, these roles form a cooperative framework to safeguard personal data rights under GDPR.

While the GDPR has been instrumental in establishing robust protections for personal data across the EU, it is becoming increasingly clear that this framework alone may not be sufficient to address the rapid advancements in technology. Emerging technologies, such as large language models, pose unique challenges that require ongoing regulatory updates to ensure that privacy, safety, and ethical standards are maintained. The recent EU AI Act ⁸, which took off in August 2024, represents a significant step toward addressing these challenges. This regulation introduces a risk-based approach, categorizing AI systems based on their potential to cause harm, with stricter regulations for high-risk

⁸AI Act: https://commission.europa.eu/news/ai-act-enters-force-2024-08-01_en

applications (e.g., medical AI, recruitment systems) and outright bans on unacceptable risks, such as AI-enabled social scoring or certain types of biometric surveillance. This evolution underscores the need to stay up-to-date with regulatory developments as new risks emerge. Despite these advancements, enforcing such regulations remains complex and resource-intensive. Balancing the demands of technological innovation with regulatory safeguards presents a major challenge, as tighter controls may squash innovation, while lax enforcement could undermine safety and privacy. Finding this balance will be crucial as we navigate the expanding role of AI and data-centric technologies in our societies.

2.8 Software Licenses and IP Regulations

Software licenses define the terms under which software can be used, modified, shared, and distributed. They are crucial in determining how software interacts with users, developers, and businesses. Figure 2.9 summarizes the rights granted in the context of the copyright for the main software licenses commonly used [80].

Proprietary Software Licenses: Proprietary software licenses are restrictive and generally prevent users from accessing or modifying the source code. Users typically purchase or subscribe to proprietary software, and the rights to modify or redistribute the software are retained by the creator or company. A common example are commercial videogames, or Microsoft Windows, which users can utilize but not alter or redistribute.

Open-Source Licenses: On the other hand, open-source licenses allow users to freely access, modify, and distribute the software’s source code, although the conditions vary depending on the specific license. Open-source licenses can be divided into permissive and copyleft categories. Permissive licenses like the MIT License and Apache License 2.0 impose few restrictions on how the software can be used or incorporated into other projects, including proprietary ones. This flexibility encourages innovation and integration with various software ecosystems. Conversely, copyleft licenses, such as the GNU General Public License (GPL), require that any derivative works or modified versions of the software be distributed under the same license. This ensures that the software

Rights Granted	Public domain	Permissive license (eg.,BSD license)	Copyleft license (e.g., GPL)	Freeware Shareware Freemium	Proprietary License
Copyright Retained	No	Yes	Yes	Yes	Yes
Right to Perform	YES	Yes	Yes	Yes	Yes
Right to Display	YES	Yes	Yes	Yes	Yes
Right to Copy	Yes	Yes	Yes	Often	No
Right to Modify	Yes	Yes	Yes	No	No
Right to Distribute	Yes	Under some license	Under some license	Often	No
Right to Sublicense	Yes	Yes	No	No	No
Example Software	SQLite, ImageJ	Apache web server, Toy Box	Linux Kernel, Gimp, OBS	Irfanview, Winamp	Windows, Spotify, commercial video games

Figure 2.9: Software licenses and rights granted in the context of the copyright according to Troan [80].

and any enhancements remain open and freely available to all. The Lesser GPL (LGPL) provides a more permissive option by allowing the software to be used in proprietary projects, as long as certain conditions are met.

Public Domain Licenses: In addition to these widely used licenses, some software is placed in the public domain, meaning it is not subject to copyright protection and can be freely used, modified, and distributed by anyone. This is the most permissive form of licensing, exemplified by licenses like Creative Commons Zero (CC0), which dedicates software to the public domain. Another notable approach is dual licensing, where the software is offered under both an open-source license and a proprietary license. This model allows developers to cater to both open-source communities and commercial users. An example is MySQL, which can be used freely under the GPL but also offers a commercial license for companies that want to avoid the obligations imposed by the GPL.

Freeware and Shareware: Furthermore, terms like freeware and shareware describe software that is distributed at no cost or with limited functionality. Freeware, such as

Adobe Acrobat Reader, is often available for free without access to its source code, while shareware, like WinRAR, offers a trial period or limited functionality before requiring payment for continued or full-featured use. Unlike open-source software, freeware and shareware typically do not grant users the ability to modify or redistribute the software.

Software license infringements occur when the terms of a software license are violated, leading to legal or financial consequences for individuals or organizations involved. Infringements can range from unauthorized copying, distribution, or modification of software to improper use of open-source code in proprietary projects without adhering to the license’s terms. For example, in copyleft licenses like the GPL, failing to release the source code of derivative works or modifications violates the license, resulting in potential legal action [55]. In proprietary software, common violations include using pirated copies, exceeding the number of allowed installations, or using the software outside its licensed scope. Companies or developers may face lawsuits, fines, and injunctions for these infringements, and in some cases, the software could be barred from use or further distribution. In the open-source community, while legal actions for infringements are less common, developers can still enforce the license terms to protect their intellectual property. Understanding and adhering to the license terms is crucial to avoid these infringements, ensuring that software is used and distributed in compliance with legal agreements [55].

2.8.1 License Compliance and Code Language Models

In the context of copyright law, the protection of code presents unique challenges compared to other forms of creative works such as text or music [55]. Unlike natural language text, where the boundaries between fair use and infringement are relatively well-established, software copyright law remains more nuanced and unsettled. The difficulty arises primarily from the dual nature of software as both a functional and creative work. While literal copying of text may constitute copyright infringement, the non-literal elements of code such as algorithms, structures, and the overall functionality often occupy a gray area in copyright law [55]. Courts have acknowledged this ambiguity, recognizing that the copyrightability of non-literal aspects like the architecture or sequence of a

program is not always clear-cut [55].

This complexity makes it challenging to determine the extent to which code is protected, particularly when CLMs, are involved in generating and reproducing code. Despite their remarkable capabilities, CLMs often overlook critical license information when reproducing open-source code. This has ignited widespread concern within the open-source community, as it raises both ethical and legal issues. For instance, a prominent open-source developer has accused GitHub Copilot of “emitting large chunks of my copyrighted code, with no attribution, no LGPL license”⁹. These concerns go beyond ethical considerations; they also introduce significant and unpredictable legal risks for users. Specifically, when LLMs generate open-source code without the appropriate licensing details, users are unable to ensure compliance with legal requirements. This lack of attribution can expose users to potential copyright infringements, with substantial legal and financial consequences [81].

Currently, OpenAI, GitHub, and Microsoft face ongoing lawsuits related to GitHub Copilot, which has been accused of reproducing licensed code without adhering to the terms of the respective licenses¹⁰. Such conflict highlights the growing concern surrounding LLMs and their ability or failure to respect open-source licenses during code generation. The risks of non-compliance are particularly concerning for businesses and organizations, which may inadvertently expose themselves to legal liabilities, reputation damage, and costly settlements [81].

Open-source software authors typically grant rights to their code through specific licenses, which establish clear conditions for reuse. These licenses are integral to the open-source standards, allowing others to use, modify, and distribute code, provided that the conditions set forth by the license are followed. Crucially, open-source authors are not opposed to reuse their code; rather, they welcome it, as this aligns with the very principles underpinning the open-source movement [81]. However, compliance with license terms such as attribution, inclusion of license text, or the sharing of modifications remains a core requirement. Failure to meet these obligations results in non-compliance,

⁹T. Davis. (2022, October): <https://twitter.com/DocSparse/status/1581461734665367554>

¹⁰GitHub Copilot litigation: <https://githubcopilotlitigation.com/>

potentially leading to intellectual property violations and legal disputes [55], [81].

The ongoing controversies surrounding LLM-generated code underscore the importance of ensuring license compliance [81]. The ability of LLMs to accurately provide license and copyright information during code generation is essential, not only to protect the intellectual property rights of open-source developers but also to safeguard users of these models from unforeseen legal challenges. As the use of LLMs in code generation grows, addressing this issue becomes critical to fostering a legally compliant and ethical ecosystem for open-source software.

Chapter 3

Related Work (SOTA)

This chapter presents a comprehensive review of recent and relevant research on memorization in large language models, with particular emphasis on code models aimed at enhancing the reader’s understanding of the current SOTA. It discloses topics such as membership inference attacks, both targeted and untargeted data extraction attacks, and recent findings on fine-tuning memorization. By exploring these key areas, this chapter aims to provide a robust contextual framework for understanding prior work in the field, thereby highlighting the research gaps that this study seeks to address.

3.1 Memorization

In the context of language models, many have registered models’ ability to recall and regurgitate substantial parts of the data it has been trained on [3], [7], [12], [16], [82], [83]. This phenomenon is identified as *memorization*, and recently the study of this occurrence has gained attention and interest from the AI research field considering its implications for model performance privacy and security. According to Carlini *et al.* memorization occurs when a model overfits the training data, meaning it becomes overly specialized and fails to generalize well to new or unseen data.

Hartmann *et al.* [17] classified different types of memorization. The most direct and low-level form of memorization [84] is verbatim memorization, which is the tendency of the model to regurgitate entirely or partially the training documents. However, that’s not

the only form of memorization, a model can recall facts, mimic writing styles, or follow properties of the training distribution [17]. Memorization is an intrinsic process in LLMs, and as Hartmann *et al.* highlight, different types of memorization have distinct positive and negative implications depending on their application. These implications, including their impact on performance, privacy, security, and ethical concerns, are summarized in Table 3.1.

Verbatim text memorization can improve the model’s ability to generate accurate quotes or reuse important phrases, which is beneficial for tasks like book reviews or generating legal documents. However, it can also lead to unintended consequences, such as the leakage of proprietary or sensitive information, which raises concerns in fields like cybersecurity and data privacy [17].

Facts, on the other hand, are crucial for tasks that require world knowledge, but when LLMs memorize personal details or proprietary data disguised as factual information, they can inadvertently leak sensitive details such as API keys or passwords [17].

Ideas and algorithm memorization enhance the model’s ability to solve complex tasks by remembering common algorithms or procedural steps, which can boost performance in areas like programming assistance or computational problem-solving. However, this also risks copying intellectual property or propagating harmful or incorrect ideas, especially if the LLM is trained on biased or inaccurate data [17].

LLMs can mimic the writing style of specific authors or formal tones. Yet, this could also lead to impersonation risks, social engineering attacks, or even copyright violations if the model reproduces a copyrighted style without permission [17].

Memorizing the properties of the training distribution helps LLMs generalize across varied tasks, but it also raises concerns about bias, as memorizing distributional biases from the training data could lead to reinforcing stereotypes or marginalizing underrepresented groups [17].

Alignment goals, which are infused through human feedback or ethical guidelines, are necessary to ensure the model behaves according to human values. However, these goals could also leak information about the preferences and biases of the human labelers involved in the training process, potentially compromising the model’s neutrality and

Table 3.1: Main potential implications of the kinds of memorization covered by Hartmann *et al.* across different domains [17]. Positive implications are marked with (+), and negative implications are not marked.

Type of Memorization	Performance, Alignment	Privacy	Security, Confidentiality	Copyright
Verbatim text	capability to quote (+), ignoring instructions	leaking sensitive data information	leaking proprietary training data	regurgitating
Facts	answering questions (+), overriding prompt data	leaking personal information	leaking crypto- graphic keys	-
Ideas and algorithms	performing common algorithms (+), reproducing harmful ideas	decontextualization of irony	leaking secret ideas	violation through fan fiction
Writing styles	style transfer (+), reproducing toxicity	author attribution	social engineering	violation in combination with general idea of a work
Properties of training distribution	necessary for learning (+)	identifying human labelers	theft of preprocessing parameters	help in identifying potential violations (+)
Alignment goals	necessary for alignment (+)	leaking preferences of human labelers	spotting weaknesses in safety alignment	-

fairness [17].

3.2 Memorization in Code Language Models

Recent studies have highlighted this risk by CLMs of regurgitating training data, some of which might contain sensitive information [3], [12]. For example, GitHub Copilot and Amazon CodeWhisper can be coaxed to emit hardcoded credentials captured during training, using so-called data extraction attacks [9]. The issue of memorization is especially concerning in code models and should warrant careful attention from the research community.

Yang *et al.* identify three critical scenarios in which memorization can pose significant risks to code models. Firstly, many code models are trained on publicly available repositories from platforms like GitHub, often without adequately considering the licensing terms associated with the data. As a result, model memorization can lead to intellectual property violations, including the infringement of open-source licenses. If a model reproduces specific code snippets from its training data rather than abstracting underlying programming principles, it risks generating outputs that closely mirror copyrighted or

licensed code, infringing on the original authors’ intellectual property rights [7], [12], [55].

Secondly, training datasets for code models may inadvertently contain vulnerable or malicious code. When a model is trained on such data, there is a risk that it could generate code with security flaws, propagate insecure coding practices, or even produce malicious or exploitable code. The memorization of such insecure code poses substantial security threats to users and systems relying on these models, as it may lead to the unintentional dissemination of vulnerabilities [7].

Lastly, public repositories often contain sensitive information, including software secrets that can inadvertently be included in training datasets. If a code model memorizes such data, it may reproduce these secrets, leading to the unintentional leakage of sensitive information. This risk becomes particularly concerning when considering the scale of data ingested by CLMs, highlighting the need for robust safeguards to prevent the exposure of confidential information [5], [7].

3.3 Membership Inference Attacks

Memorization enables membership inference attacks [16], [82], [85]. We refer to the security game defined by Carlini *et al.* [16] to define a membership inference attack in Definition 1.

Definition 1 (Membership inference security game [16]) *The game proceeds between a challenger $\mathcal{C}$, an adversary $\mathcal{A}$, a data distribution $\mathbb{D}$ and a model f :*

1. *The challenger samples a training dataset $D \leftarrow \mathbb{D}$ and trains a model $f_\theta \leftarrow \mathcal{T}(D)$ on the dataset D .*
2. *The challenger flips a bit b , and if $b = 0$, samples a fresh challenge point from the distribution $(x, y) \leftarrow \mathbb{D}$ (such that $(x, y) \notin D$). Otherwise, the challenger selects a point from the training set $(x, y) \leftarrow D$.*
3. *The challenger sends (x, y) to the adversary.*

4. The adversary gets query access to the distribution $\mathbb{D}$, and to the model f_θ , and outputs a bit $\hat{b}$
5. Output 1 if $\hat{b} = b$, and 0 otherwise.

In this game, the adversary wins if they have a non-negligible advantage $> \frac{1}{2} + \epsilon$. In simpler terms, the adversary needs to be able to distinguish between data that was included and which was not included in the training data for a given model, while only being allowed query access to the model and data distribution.

Shokri *et al.* proposed the first membership inference attack in which the authors targeted classification models [86]. Since then the field has expanded and proposed attacks target generative models, including LLMs [13], [16], [87] and vision models [88].

Membership inference attacks are primitive for measuring the leakage of a machine learning model and are often a starting point for more extensive attacks [13], [16], [83]. While membership inference is a weaker privacy violation than memorization, the National Institute of Standards and Technology (NIST) still considers membership inference to violate the confidentiality of training data [83].

In the code domain, an additional risk is raised by the presence of copyleft and other non-permissively licensed codes in training datasets [12]. Katzy *et al.* found that datasets used to train code LLMs contain non-permissively licensed code, even those that claim otherwise [56]. Wan *et al.* used membership inference to detect if code was used in the training of LLMs [89].

3.4 Untargeted Data Extraction Attacks

Data extraction attacks are a stronger type of attack where an adversary extracts a data point used to train a model. Attacks can be divided into two kinds for LLMs, untargeted and targeted attacks. Both are schematized in Figure 3.1.

Yang *et al.* studied memorization in *pre-trained* code LLMs using an untargeted attack against CodeParrot identifying several key factors of memorization and giving suggestions to deal with memorization [7].

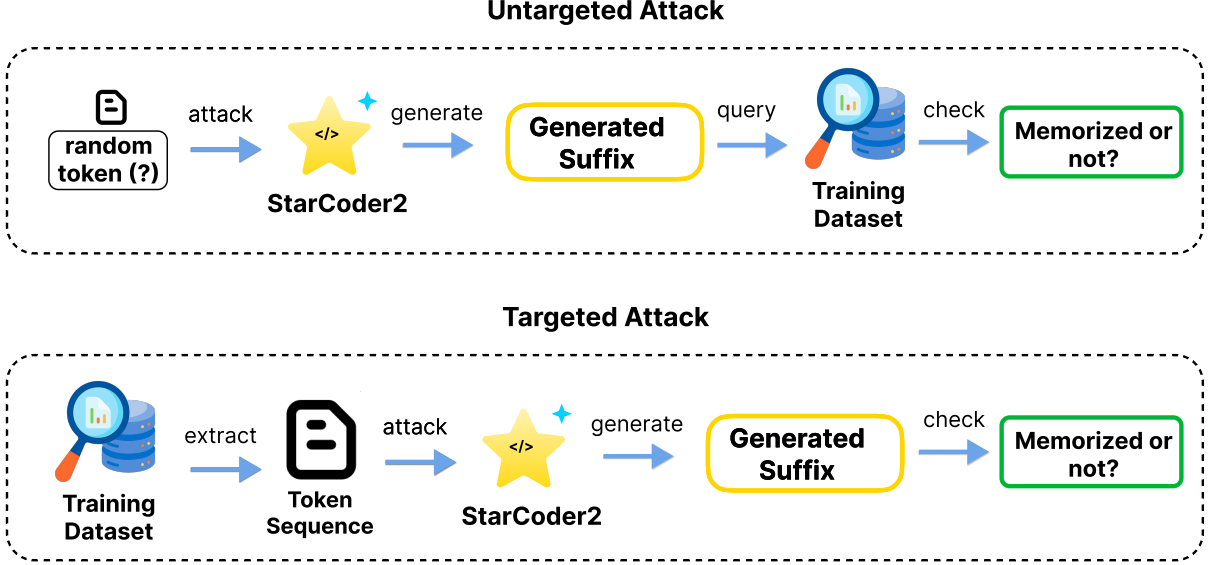


Figure 3.1: Untargeted and targeted data extraction attacks.

In untargeted attacks, the model is prompted with an empty string or a random token. The completion of the model is then checked for memorized sequences. The adversary does not know the sample to be extracted from the model, it simply attempts to extract any training points. The memorized sequence can be from any part of the training corpus [3], [82], [87]. This attack requires knowledge of the complete training data, as any random sequence can be emitted, as there is no control of the target [13], [55], [85], [90].

3.5 Targeted Data Extraction Attacks

Carlini *et al.* first proposed a targeted data extraction attack and demonstrated that LMs tend to memorize and regenerate segments of training data [3]. Biderman *et al.* proxies verbatim memorization defining the concept of *k-extractability* which was originally inspired by the framework of *k-eidetic* memorization introduced by Carlini *et al.* [3].

Definition 2 (k-extractability [4]) *A string s is said to be k -extractable if it (1) exists in the training data, and (2) is generated by the language model by prompting with k prior tokens.*

Targeted data extraction attacks consist of extracting pre-defined training sequences by splitting the string sequence from the training data into a k-tokens prefix and a suffix, aiming to generate the suffix from the model by attacking it with the k-tokens prefix. In a targeted attack, the model is supplied with partial information to complete. Targeted attacks are more security-critical as they allow for targeting specific information. This kind of attack requires knowledge of a portion of the training data.

Kandpal *et al.* and Lee *et al.* revealed that duplicated training data is more vulnerable to memorization, and de-duplication can effectively reduce memorization [91], [92]. Carlini *et al.* further quantified memorization effects, exhibiting that memorization grows with model scale, data duplication, and prompt input length [84].

Al-Kaswan *et al.* also measured memorization in *pre-trained* code LLMs with a targeted attack. By sampling common code from *Python* files and identifying potentially extractable samples, the authors constructed a benchmark dataset identifying code licenses and code documentation as the most vulnerable information to be extracted [5].

3.6 Memorization after Fine-tuning

Many code LLMs are deployed using the pretrain-finetune paradigm. Under this paradigm, an LLM is first pre-trained with a large unlabelled dataset in an unsupervised manner. Then, the LLM is fine-tuned with a smaller task-specific dataset [37], [66]. Models such as CodeT5 are first trained on a large corpus of source code and then further fine-tuned for specific tasks such as code summarization and code completion [66]. Although state-of-the-art language models, such as StarCoder2 [8], CodeLlama [60] are proficient in a variety of tasks without requiring any fine-tuning, there are other reasons to fine-tune a model. Other use cases for fine-tuning include adapting a model to a specific project [93], organization, or a different field [94]. Additionally, LLMs can be fine-tuned with instruction data [95]. From the perspective of memorization, fine-tuning data can be easier to extract since it has been observed more recently by the model.

Mireshghallah *et al.* [13] conducted a comprehensive study to evaluate the effects of various fine-tuning strategies on the memorization capabilities of GPT-2, a widely

used transformer-based language model. The investigation focused specifically on how different fine-tuning methods affect the model’s propensity to memorize training data. Their findings indicate that selectively fine-tuning only the head of the model, while keeping the rest of the model’s weights frozen, leads to an increased rate of memorization of the fine-tuning dataset compared to applying full model fine-tuning. Full model fine-tuning typically involves updating all model parameters, which allows for more general adaptation across the entire network. However, the head-only fine-tuning constrains the learning to the model’s output layer, resulting in overfitting to the provided data and, consequently, higher memorization rates. This outcome highlights a critical trade-off between model adaptability and the risk of overfitting or memorization in fine-tuned models [13].

Zeng *et al.* [14] examined how various fine-tuning tasks impact the memorization rates of the T5 language model [67]. The results indicated that fine-tuning tasks exert a significant influence on the extent of memorization within the models, with notable differences emerging across different task categories. Specifically, the study identified summarization and dialogue tasks as those where memorization is most prevalent. This suggests that tasks requiring intricate, context-sensitive language processing, such as condensing extensive documents or maintaining coherent conversational exchanges, increase the likelihood of the model retaining training data verbatim [14].

Sun *et al.* [15] explore the privacy risks posed by fine-tuning the GPT-3 model, particularly the potential leakage of PII from its pre-training data. They demonstrate that even after fine-tuning, GPT-3 models can inadvertently memorize and output sensitive data, such as names and addresses, embedded in the original training corpus. This leakage, which arises from the model’s tendency to memorize rare or unique data points, raises significant privacy concerns, particularly when using publicly available datasets that may contain PII. The authors conduct experiments to quantify the risk and find that fine-tuning exacerbates the issue, especially in domain-specific tasks. They also propose mitigation strategies, such as differential privacy [96] and regularization, though they note that completely preventing PII leakage remains a significant challenge [15].

In the code domain, Pappu *et al.* investigated memorization in RLHF for code com-

pletion models [97]. Memorization of pre-training data can persist through RLHF, but the memorization rate of the RLHF data is much lower [97]. For both, the memorization rate mostly depends on the KL divergence penalty [97].

3.7 Our contribution

To the best of our knowledge, most analyses of memorization in real-world LLMs for code have been limited to the pre-training phase. In our study, we extend this investigation to examine the impact of *fine-tuning* on memorization for *various sizes* of a widely used family of open-weight and open data models. We also *manually* investigate a set of extracted data to understand the type of memorized information. While previous research primarily focused on the memorization of Python code, our work uniquely explores the memorization of *Java* code, which has not been studied before.

Chapter 4

Problem Definition

Previous studies have shown that LLMs trained on code tend to memorize parts of their training data [5], [7], [97]. Fine-tuning these models often uses small, curated datasets, which are often proprietary and valuable. This poses a risk of sensitive information being unintentionally exposed through the fine-tuned models [13]–[15]. In this work, using data extractability techniques, we investigate how much code LLMs memorize the data they have seen before and after *fine-tuning*. Figure 4.1 provides an overview of the configuration of the experiments. We adopt a setup introduced by Carlini *et al.* [82] to conduct targeted data extraction attacks. Targeted attacks are more security-critical as they allow for targeting specific information from the training corpus. We perform two types of targeted attacks: a *pre-training code attack* to evaluate the extractability of pre-training data, and a *fine-tuning code attack* to quantify the vulnerability of recent data encountered by the model.

4.1 Code Completion

In this work, we investigate memorization in fine-tuned code models, examining the phenomenon and its implications for data extractability [3], [5], [85]. We examine this phenomenon for the automatic code completion task, which has emerged as a highly prevalent and valuable application of LLMs [1]. Code completion tools have become integral to modern software development, leveraging AI to enhance productivity. A

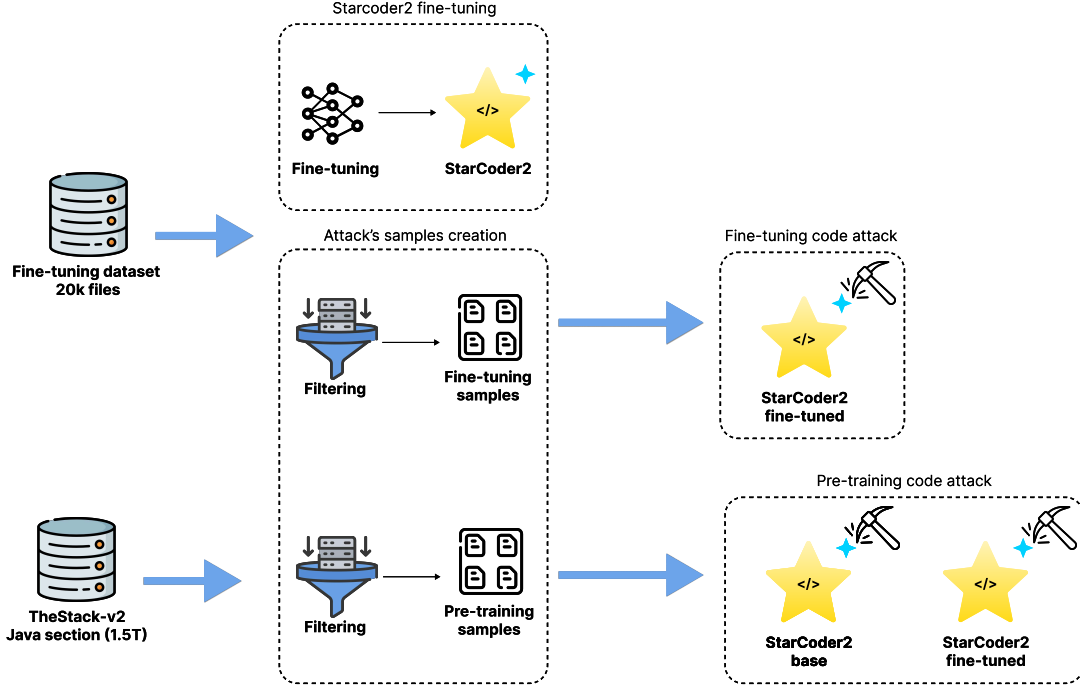


Figure 4.1: Overview of the experiments' configuration.

prevalent approach for powering code completion systems is using large language models trained via causal language modeling [2]. These models excel in generating text and, when fine-tuned on programming-specific datasets, such as publicly available code repositories, they can produce code that is both syntactically correct and stylistically consistent with the given context [2]. Code completion also represents a setting where leakage of user data can raise legal, commercial, and privacy concerns [12]. To this end, we fine-tune the StarCoder2 model [8] and its various available parameter sizes into a downstream task; line completion in Java. We focus on Java for several reasons. Java is widely used in enterprise environments, making it a relevant and practical choice. Java's well-defined syntax and structure may influence memorization, providing an interesting area of exploration. Further, previous studies on code memorization have predominantly focused on Python [5], [7], [97], leaving a gap in understanding how these phenomena manifest in other programming languages such as Java.

4.2 StarCoder2 and TheStack v2

For this study, we utilized StarCoder2 [8]. StarCoder2 is a transparently trained open code LLM and comes in three sizes: 3B, 7B, and 15B parameters. Developed by The BigCode Project ¹, StarCoder2 is an open-source code LLM. All models use Grouped Query Attention, a context window of 16,384 tokens with sliding window attention of 4,096 tokens, and were trained using the Fill-in-the-Middle objective [8]. StarCoder2-15B was trained on 600+ programming languages from The Stack v2 on 4+ trillion tokens [8].

The Stack v2 is the largest open code dataset suitable for LLM pre-training; it builds on Software Heritage’s extensive source code archive, which covers over 600 programming languages. In addition to code repositories, The Stack v2 includes curated data from high-quality open sources such as GitHub issues, pull requests, Kaggle and Jupyter notebooks, code documentation, and other natural language datasets related to mathematics, coding, and reasoning. The complete dataset comprises approximately 67.5 TB and around 900 billion tokens. ² We chose StarCoder2 as it is one of the most recent open-source code models. StarCoder2 and its derivatives are the best-performing models with both open weights and open data on the EvalPlus Leaderboard [18]. ³ Its open-data nature provides us with complete access to the training corpus, which is essential for conducting targeted attacks and assessing the extractability of pre-training data [5], [7].

4.3 Research Questions

RQ1: To what extent are pre-training data extractable from pre-trained models? We conduct a data extraction attack on the base version of StarCoder2 to evaluate the extractability of its training data. We examine two factors that might influence extractability: model size and input prefix length [3], [5].

RQ2: To what extent are pre-training data extractable from fine-tuned mod-

¹The BigCode project organization: <https://www.bigcode-project.org/>

²TheStack v2: <https://huggingface.co/datasets/bigcode/the-stack-v2>

³EvalPlus Leaderboard: <https://evalplus.github.io/leaderboard.html>

els? After fine-tuning the model on the Java line completion task, we repeat the pre-training code attack to compare the extractability before and after fine-tuning.

RQ3: To what extent are fine-tuning data extractable from fine-tuned models?

We conduct a data extraction attack on the fine-tuned StarCoder2 to evaluate the extractability of recent fine-tuning data. Factors analyzed include model size, training epochs, duplication rate of samples, and input prefix length variations [3], [5].

RQ4: Does the type of code data impact extractability? Finally, we investigate whether certain data categories are more or less extractable after fine-tuning. We analyze samples from both fine-tuning and pre-training data extraction attacks, constructing a classification to evaluate their extractability.

Chapter 5

Methodology

This chapter outlines the methodologies utilized in this study, detailing the processes of dataset construction, and model fine-tuning. It also describes the data extraction technique used to retrieve the attack samples and execute the data extraction attacks. Furthermore, we provide a comprehensive overview of the code categorization approach employed to analyze the different code data types.

5.1 Fine-Tuning

5.1.1 Dataset

The fine-tuning dataset consists of 20k `.java` files randomly sampled from a curated Java dataset [98]¹. This dataset was created by scraping 10.5k public repositories on GitHub. The focus was on repositories released under strong copyleft licenses, as these licenses are excluded from the StarCoder2 training corpus [8], [56]. The dataset creation pipeline involves three key stages: collection, cleaning, and deduplication. The first stage entails scraping public GitHub repositories and extracting all Java files. The next stage involves cleaning the dataset. Finally, the last stage is the deduplication process, addressing both exact and near-duplicates between the curated Java dataset and the Java-Stackv2. Figure 5.1 schematizes the pre-processing conducted and the following

¹Java Dataset: https://huggingface.co/datasets/LaughingLogits/Stackless_Java_V2

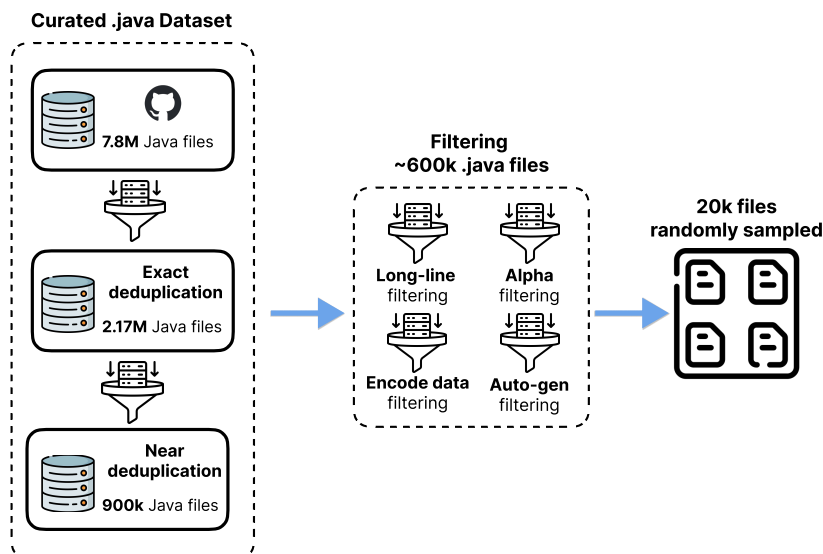


Figure 5.1: Fine tuning dataset construction pipeline.

paragraphs provide a detailed explanation of each stage.

The collection process involves scraping 10.5k public repositories created between 2001 and 2023 using the GitHub API. The focus was on repositories released under a strong copyleft license such as GPL-2.0, GPL-3.0, or AGPL-3.0 [98]. The reason for this is to determine if there are any code license infringements in Stackv2 they claim to exclude copyleft license in their paper, as was previously found with other publicly available file-level code datasets, including Stackv1, which claimed to contain only permissive licenses [56]. Once the repositories were retrieved, it was extracted all the files with a .java extension.

The next stage in the dataset creation pipeline is the cleaning process. Removed all the large Java files, over 50 MB, and the ones with less than 10 words. The large files can lead to excessive memory usage, which could slow down the pre-processing, and also contain irrelevant data which might lead to overfitting. On the other side, really short files may lack sufficient context and informative content, which can provide minimal learning value for the model [98].

The final stage of the dataset pipeline is the deduplication process. The scraped files went through an exact deduplication process against Java-Stack-v2, as well as between the scraped Java files themselves. The technique employed involved using the

sha256 function to calculate the hashes for each dataset [98]. Lastly, the scraped files went through a near-deduplication process against the Java-Stackv2 using the MinHash Locality Sensitive Hashing (MinHashLSH) algorithm from the data-sketch library. Minhashes are calculated with the sha256 hash function from which are extracted the first 16 bytes to generate a 128-bit hash value. Lastly, a Jaccard similarity threshold of 0.7 is used [98], this approach proved to be efficient for both SantaCoder and StarCoder models [59], [64].

Before sampling, we applied several filters following the training data pre-processing pipeline implemented for StarCoder2 [8]. These filters include a long-line filter, an alpha filter, an encoded data filter, and an auto-generated filter.

- *Long line filters*: First, we excluded files containing more than 100k lines, as these are likely to be either data files or automatically generated code. We also filtered out files with an average line length exceeding 100 characters or with a maximum line length of more than 1,000 characters, as such files may not be representative of the typical source code structure [8].
- *Autogenerated filter*: To eliminate auto-generated files from our dataset, we apply the `is_generated` function of `go-enry`². Additionally, we filter out files that contain any of the following terms within the first five lines: {"auto-generated", "autogenerated", "automatically generated", "generated automatically", "this file is generated"} [8].
- *Alpha filter*: We remove files with less than 25% of alphabetic characters [8].
- *Encoded data filter*: For detecting files with inline encoded data, we use the following regular expressions which were designed by Lozhkov *et al.* [8]:
 - Base64 strings: `[a-zA-Z0-9+/\n=]{64,}`
 - Hexadecimal sequences: `(?:\b(?:0x|\x)?[0-9a-fA-F]{2}(?:,|\b\s*))\{8,}`
 - Unicode strings: `(?:\\u[0-9a-fA-F]{4})\{8,}`

²GitHub go-enry: <https://github.com/go-enry/go-enry>

We remove the file if any of the substrings matching these expressions is longer than 1024 characters or if the fraction of matched characters is more than 50% of the file [8].

All the filter’s implementations can be found inside the replication package [19].

5.1.2 Training

The fine-tuning of each model was conducted using the resources provided by the Delft High-Performance Computing Centre [99]. We train the models on an HPC cluster. We utilize 32 CPU cores with 32GB of RAM and a set of Nvidia A100 GPUs, each with 80GB of memory. The experiments are performed on various sizes of the StarCoder2 model, with the specific number of GPUs allocated based on the model’s parameter count to fit within memory constraints. Specifically, for StarCoder2-3B, 7B, and 15B, we employ two, four, and six GPUs, respectively. This results in approximate training times of 25, 55, and 110 hours, respectively. The GPUs are running Nvidia driver version 555.42.02 and CUDA 12.5. The training utilizes Transformer version 4.41.1 on Torch 2.3.0+cu121.

To ensure consistent comparison across the different sizes of StarCoder2, we maintain the same training setup for each model size. The effective training batch size (including gradient accumulation) is 24, 24, and 25, respectively. The context window is set to 1024 tokens. We use a learning rate of $3e - 5$ and utilize a linear scheduler and the Adafactor optimizer [100] to reduce GPU memory requirement and to make reproducibility easier.

Each model was trained for three epochs, with checkpoints saved at the end of each epoch. We identified GPU memory and training time as limiting factors, influencing our choice of optimizer, training files, and batch size.

5.2 Data Extraction Attacks

We first introduce a precise definition of memorization proposed by Carlini *et al.* [84].

Definition 3 (memorization) *A string s is extractable with k tokens of context from a model f if there exists a (k -length) string p , such that the concatenation $[p||s]$ is contained*

in the training data for f , and f produces s when prompted with p using greedy decoding.

This is defined as verbatim memorization. Ippolito *et al.* proposed a relaxed definition of memorization using the BLEU score [101]. To measure memorization, we first formally define the data extraction game and then describe the approach used to obtain the pre-train and fine-tuning code samples for performing the data extraction attacks.

5.2.1 Data Extraction Security Game

In our experiments, we consider the models as black-box systems. We define a security game inspired by the membership inference attack security game and the notion of k -extractability [3], [5], [84].

Definition 4 (Data extraction security game) *Given a challenger $\mathcal{C}$, an adversary $\mathcal{A}$, a data distribution $\mathbb{D}$ and a model f the game is defined as follows:*

1. *The challenger samples a training dataset $D \leftarrow \mathbb{D}$ and trains a model $f_\theta \leftarrow \mathcal{T}(D)$ on the dataset D .*
2. *$\mathcal{C}$ samples a sample $D_n = (p, s)$ where $D_n \in D$. The prefix p is provided to the adversary $\mathcal{A}$.*
3. *$\mathcal{A}$ is allowed query access to the model f_θ and may perform any other polynomial-time operations*
4. *$\mathcal{A}$ outputs his prediction sequence $\hat{s}$*
5. *If $\hat{s} = s$, $\mathcal{A}$ wins, otherwise $\mathcal{C}$ wins*

In other words, given a prefix p (1), the adversary is challenged to extract the correct suffix from the model’s training data. The adversary can query the language model f_θ (2) but cannot inspect the weights or hidden states (e.g., attention vectors). The adversary then predicts the suffix s (3) and wins if it matches the actual suffix in the training data [3], [5], [84].

Some factors may affect the difficulty of the challenge:

1. The selection of the sample $D_n \in D$. As noticed by previous works, not all training samples are as hard to extract as others. In particular, highly duplicated or outlier samples are more vulnerable to attacks [84], [91], [92].
2. The size of the model f_θ . Some models are more likely to memorize samples than others, i.e., larger models have been observed to memorize more pre-training data than smaller models [5], [7], [84].
3. The length of the prefix p . It has been found that longer prefixes induce more memorization. [7], [84].
4. The victory condition $\hat{s} = s$. Using the exact match might exclude samples that are entirely extractable but have meaningless differences e.g., a space character. So, a fuzzy match can also be considered [101].

In this work, we exploit the security game and the factors (1,3,4) described above to construct a set of extractable samples to evaluate different model sizes (2). We also measure fuzzy match scores (4) and compare them with the exact match rate.

5.2.2 Data Extraction Dataset

To measure memorization in LLMs for code, we first need to construct a dataset analogous to the one used in the SaTML23 Language Model Data Extraction Challenge.³ We can divide the extractions into *pre-training code attack* (extracting pre-training data from the model) and *fine-tuning code attack* (extracting fine-tuning data from the model). In our experiments, we use different sizes of StarCoder2 to improve generalizability.

Pre-training code attack dataset: We collect the pre-training samples for the attack from the Java subset of TheStack-v2 through the following procedure. The candidate samples for the attack are obtained by randomly selecting 300 token spans from anywhere in the file taking into account the context window used to train StarCoder2 [8]. All the files with less than 300 tokens were discarded. We considered only candidates with

³Language Models Training Data Extraction Challenge: <https://github.com/google-research/lm-extraction-benchmark>

more than three duplicates inside the training corpus to be consistent among the RQs given the space limit. Additionally, to avoid overlaps we exclude the candidate samples also included in the fine-tuning set. We randomly select 1K samples from the filtered candidate set to perform our evaluation. Finally, we split the 300 tokens sequences into a suffix of 50 tokens and a prefix of 250 tokens. In our experimental setup, we will truncate the prefix to 100, 150, and 200 tokens to test extractability when feeding the inputs with different prefix lengths to the models. As in Al-Kaswan *et al.* [5], our pre-train attack dataset construction procedure differs from the one used by Carlini *et al.* [84] in one aspect. Our dataset does not guarantee that for every $D_n = (s, p)$ there does not exist a $(s, p') \in D$ where $p' \neq p$. This is because the computational cost of identifying all unique samples $D_n = (s, p)$ is extremely large for a dataset of this size [5].

Fine-tuning code attack dataset: The approach for collecting fine-tuning samples for the attack presents variations from the previously described procedure. Given the smaller scale of the fine-tuning set compared to the pre-training set, we implemented certain modifications that enabled further experimentation. We apply the candidate samples for the attack by applying a 300 tokens sliding window to each file of the fine-tuning set while considering the context window used in the fine-tuning setup. The size of the fine-tuning set allows us to know and control the number of times a 300-token sample is in the fine-tuning set; this enables us to control the duplication rate. We divided candidate samples into subsets based on their duplication rate: unique samples ($d = 1$), single duplicates ($d = 2$), double duplicates ($d = 3$), and samples with more than three duplicates ($d > 3$). This choice is guided by inspecting the duplication rate sample distribution on the fine-tuning set. For each duplication rate subset, we randomly select 1K samples from the filtered candidate set to perform our evaluation. Finally, we split the 300 token sequences into a suffix of 50 tokens and a prefix of 250 tokens with the same approach we used for the pre-training attack dataset. The smaller scale of the fine-tuning set compared to the pre-training set allowed us to guarantee that for every $D_n = (s, p)$, there does not exist a $(s, p') \in D$ where $p \neq p'$ as in the attack dataset construction procedure by Carlini *et al.* [84].

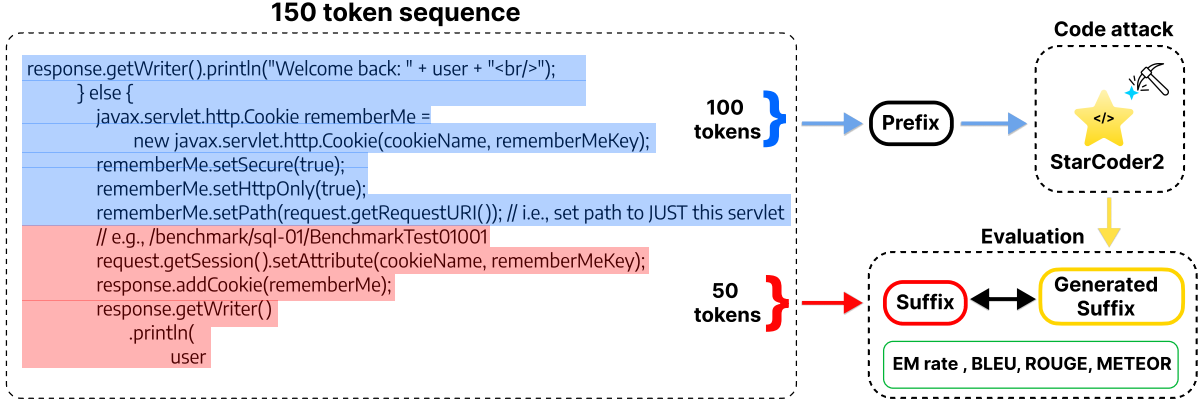


Figure 5.2: Visualization of the code data extraction attack.

5.2.3 Performing the Attacks

We prompt the model under investigation with the prefix. We utilize the standard generation pipeline and the default generation configuration of StarCoder2, as defined in its model configuration. Figure 5.2 schematizes the data extraction attack. The models are prompted with prefix sequences in a one-shot fashion with greedy decoding. The generated suffix is then compared with the true suffix, which serves as the ground truth. We evaluate the performance using two types of metrics. First, we measure the Exact Match (EM) score. Additionally, we also assess the match by fuzzier metrics such as BLEU [102], METEOR [103], and ROUGE-L [104].

5.3 Analysis through Categorization

Following the framework in the literature [5], we conduct an exploratory study to better understand the extracted data by categorizing both the pre-trained and fine-tuned attack datasets. The different categories are outlined in Table 5.1. For fine-tuning, we selected one of the four 1k code attack datasets, specifically choosing the sample set with a duplication rate of 3. This choice represents a balance between duplication rate and category representativeness.

To simplify the classification process, samples with dual purposes (e.g., samples that start with license information and then finish with code.) were assigned to their predom-

Table 5.1: Categories of data extraction datasets.

Category	Purpose	Pre-training	Fine-tuning
Code	Code Logic	388	223
Dicts	Dictionaries or other data carriers	164	117
Docs	Documentation	173	291
License	License information	124	284
Testing	Test Code	50	2
Total		899	917

inant category [5].

The author and the co-supervisor independently conducted the classification and compared their results subsequently. Before the labeling process, both reviewers jointly examine 20 samples to familiarize themselves with the dataset. The labeling process shows strong agreement between annotators, with Cohen’s Kappa scores of 0.87 for the pre-training code attack dataset and 0.90 for the fine-tuning code attack dataset. Analysis of the confusion matrix reveals that discrepancies primarily arise from differences between test categories and code categories, as well as from the decisions made when categorizing samples with dual purposes. We discard the samples that the reviewers do not agree on, and we are left with 899 and 917 samples for the pre-training and fine-tuning datasets, respectively.

Chapter 6

Results

We present the results of our experiments to answer the research questions, results are organized per research question.

6.1 RQ1: Extracting Pre-training Data from Pre-trained Models

The results of the pre-training code attack on the pre-trained models are shown in Table 6.1. We observe that increasing the length of the input prefix in attacks enhances the extractability of pre-training data from pre-trained models. We are able to extract 45.5% of the samples from the StarCoder2-15B model using a 100-token prefix. This extraction rate increases to 50.6% with a 150-token prefix, and further to 54.9% with a 250-token prefix. This pattern is also supported by analyzing the results of fuzzy metrics. The suffixes generated by the StarCoder2-15B model reached a BLEU score of 65.2% with a 100-token prefix, which improved to 72.8% with a 250-token prefix. As shown in Figure 6.1, the increase in extractability is notably more pronounced when extending the prefix length from 100 to 150 tokens compared to the increase from 200 to 250 tokens. This suggests a nonlinear growth trend concerning prefix length.

We observe that model size impacts pre-training extractability in pre-trained models. Table 6.1 indicates that using a 100-token prefix, we can extract 38.9% of the pre-training

Table 6.1: Pre-training code attack on the pre-trained models

Model	Prefix length	Memorization rate			
		EM	BLEU	METEOR	ROUGE-L
Starcoder2-3B	100 tokens	0.389	0.652	0.706	0.682
	150 tokens	0.432	0.688	0.735	0.712
	200 tokens	0.462	0.715	0.756	0.737
	250 tokens	0.483	0.728	0.770	0.751
Starcoder2-7B	100 tokens	0.395	0.663	0.720	0.694
	150 tokens	0.452	0.701	0.756	0.733
	200 tokens	0.485	0.730	0.772	0.753
	250 tokens	0.498	0.742	0.781	0.763
Starcoder2-15B	100 tokens	0.455	0.714	0.760	0.738
	150 tokens	0.506	0.757	0.770	0.774
	200 tokens	0.536	0.777	0.806	0.794
	250 tokens	0.549	0.790	0.819	0.805

code samples from the StarCoder2-3B model. This extraction rate increases to 45.5% when using the largest version of the model, StarCoder2-15B.

RQ1: The base version of StarCoder2 memorizes pre-training data, with memorization rate scaling with model size. The extractability of this data increases with longer input prefixes, though not linearly.

6.2 RQ2: Extracting Pre-training Data From Fine-tuned Models

In Figure 6.2, we plot the EM rates for the pre-training code attack on base and fine-tuned models. We notice a considerable negative impact of fine-tuning on the extractability of pre-training data. Primarily, for the StarCoder2-15B model, we observe that after three epochs of fine-tuning, 30% of the samples that were previously extractable during the pre-training stage could no longer be extracted. This trend is consistent across different model sizes to the same extent.

Table 6.2 presents the results of experiments involving the pre-training code attack

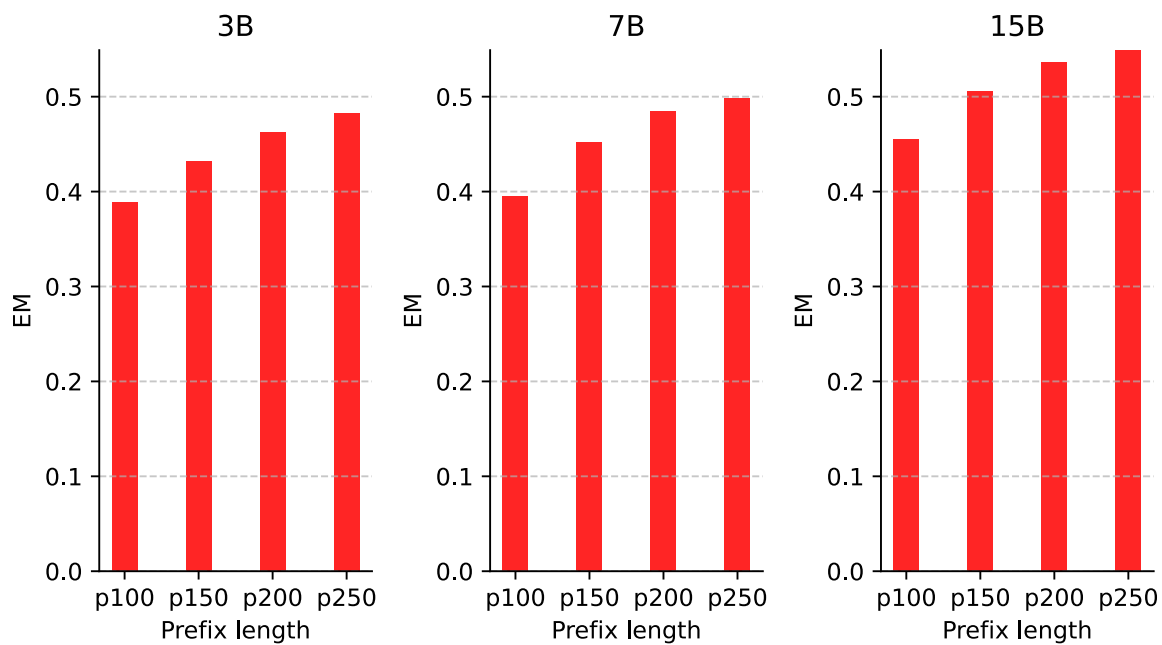


Figure 6.1: EM rate when performing pre-training code attack on pre-trained models with varying prefix lengths.

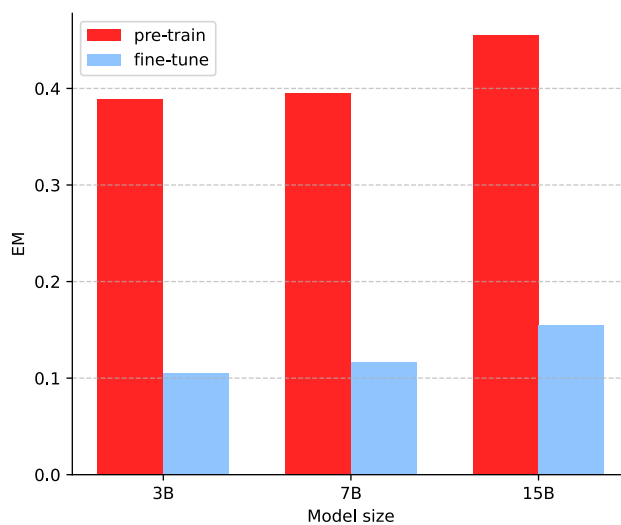


Figure 6.2: EM rate when performing the pre-training code attack (with 100 token prefix) on base and fine-tuned models.

Table 6.2: Pre-training code attack on fine-tuned models across several fine-tuning epochs.

Model	Epochs	Memorization rate			
		EM	BLEU	METEOR	ROUGE-L
StarCoder2-3B	0	0.389	0.652	0.706	0.682
	1	0.087	0.448	0.551	0.517
	2	0.108	0.452	0.552	0.518
	3	0.105	0.447	0.551	0.514
StarCoder2-7B	0	0.395	0.663	0.712	0.694
	1	0.015	0.381	0.524	0.483
	2	0.110	0.460	0.566	0.527
	3	0.116	0.470	0.572	0.533
StarCoder2-15B	0	0.455	0.714	0.760	0.738
	1	0.145	0.525	0.613	0.583
	2	0.143	0.530	0.619	0.585
	3	0.155	0.544	0.629	0.595

on fine-tuned models across the fine-tuning epochs, with the input prefix length fixed at 100 tokens. The results show that fine-tuning epochs does not significantly decrease extractability. For the StarCoder2-15B base model, we initially extracted 45.5% of the pre-training code attack dataset. This rate decreases to 14.5% after a single epoch of fine-tuning, with subsequent epochs maintaining a similar EM rate, showing only minor fluctuations within the standard error. From Figure 6.3 we can see that the StarCoder2-3B and 7B models’ extractability of pre-training data decreases after the first epoch but increases slightly in subsequent epochs. Specifically, the EM rate for the StarCoder2-3B model drops to 8.7% after the first fine-tuning epoch, increases to 10.8% by the second epoch, and stabilizes at 10.5% after the third epoch. The StarCoder2-7B exhibits a more pronounced initial decline, with the EM rate decreasing to 1.5% after the first epoch, and then increasing to 11.0% and 11.6% after the second and third epochs, respectively.

Table 6.3 shows the results of the experiments involving the pre-training code attack on the fine-tuned models while varying the input prefix length of the attacks. In line

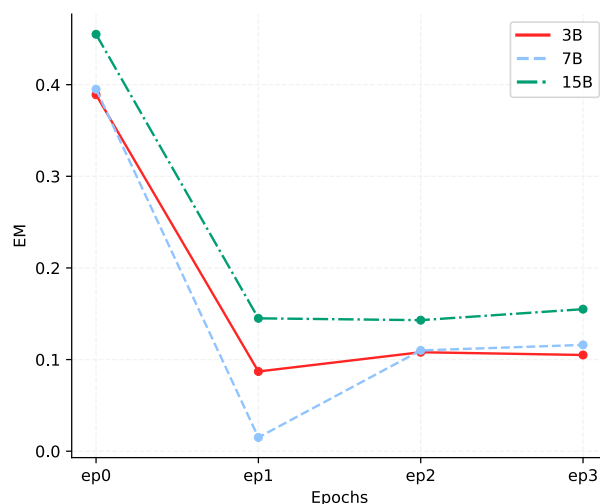


Figure 6.3: EM rate when performing the pre-training code attack on fine-tuned models (with 100 token prefixes) across fine-tuning epochs.

with the findings of the first RQ, when attacking the pre-trained models, extending the input prefix allows us to extract more pre-training samples from the fine-tuned models. For instance, with a 100-token input attack on StarCoder2-15B, we extract 15.5% of the pre-training code attack samples. This extraction rate increases to 23.5% when extending the input prefix to 250 tokens. From Figure 6.4 comparing the pre-training code attack on pre-trained models (Table 6.1) with that on fine-tuned models (Table 6.3), we observe that the increase in extractability from extending the input prefix is less pronounced in fine-tuned models compared to pre-trained models. For instance, extending the input prefix from 100 to 150 tokens allowed the extraction of 43 additional samples from the StarCoder2-3B base model. In contrast, the same prefix length extension on the fine-tuned version resulted in the extraction of only 19 additional samples.

RQ2: Fine-tuning reduces the extractability of pre-training data. This effect is consistent across model sizes and unaffected by the number of training epochs. Extending the input prefix length increases extractability but with a lower magnitude compared to pre-trained models.

Table 6.3: Pre-training code attack on fine-tuned models varying the input prefix length.

Model	Prefix length	Memorization rate			
		EM	BLEU	METEOR	ROUGE-L
Starcoder2-3B	100 tokens	0.105	0.447	0.551	0.514
	150 tokens	0.124	0.503	0.598	0.566
	200 tokens	0.151	0.526	0.613	0.583
	250 tokens	0.150	0.534	0.618	0.592
Starcoder2-7B	100 tokens	0.116	0.470	0.572	0.533
	150 tokens	0.150	0.524	0.615	0.585
	200 tokens	0.171	0.546	0.631	0.600
	250 tokens	0.178	0.560	0.642	0.612
Starcoder2-15B	100 tokens	0.155	0.544	0.629	0.595
	150 tokens	0.198	0.577	0.657	0.624
	200 tokens	0.220	0.600	0.675	0.645
	250 tokens	0.235	0.618	0.687	0.660

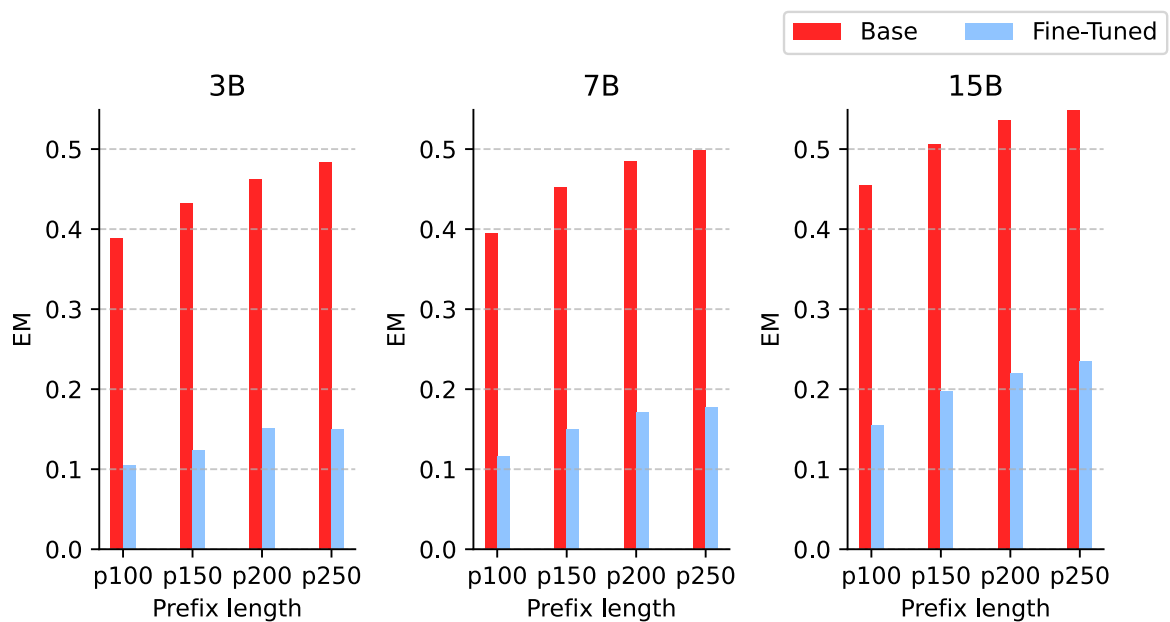


Figure 6.4: EM rate when performing the pre-training code attack on base and fine-tuned models with varying prefix lengths.

Table 6.4: Fine-tuning code attack on fine-tuned models across fine-tuning epochs

Model	Epochs	Memorization rate			
		EM	BLEU	EM $\uparrow\%$	BLEU $\uparrow\%$
Starcoder2-3B	0	0.263	0.538		
	1	0.459	0.739	0.196	0.168
	2	0.595	0.757	0.332	0.218
	3	0.631	0.764	0.368	0.226
Starcoder2-7B	0	0.249	0.546		
	1	0.408	0.739	0.159	0.158
	2	0.527	0.741	0.278	0.194
	3	0.578	0.749	0.329	0.202
Starcoder2-15B	0	0.300	0.576		
	1	0.549	0.756	0.249	0.181
	2	0.567	0.761	0.267	0.185
	3	0.609	0.774	0.309	0.198

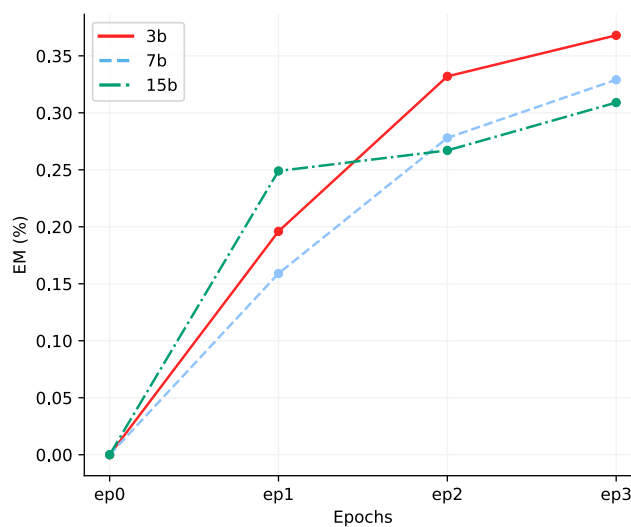


Figure 6.5: EM ($\uparrow\%$) when performing the fine-tuning code attack (with 100 token prefix) across fine-tuning epochs.

6.3 RQ3: Extracting Fine-tuning Data

In Table 6.4, we present the results of experiments involving the fine-tuning code attack across the fine-tuning epochs. Notably, performing the fine-tuning code attack on the pre-trained models revealed that some fine-tuning samples were also retrievable from the pre-trained models. In this section, we demonstrate the increase in EM and BLEU ($\uparrow\%$) relative to the baseline at epoch zero. Due to space limits, METEOR and ROUGE-L scores are available in the replication package [19]. The baseline represents the EM rate achieved by executing the fine-tuning code attack on the pre-trained models. Our experiments reveal that extractability increases with the number of fine-tuning epochs. Specifically, after one epoch of training StarCoder2-3B, we were able to extract 45.9% of the fine-tuning samples. This extraction rate increased to 59.5% after the second epoch and further to 63.1% after the third epoch.

In Figure 6.5, we present the increase of the EM rate over the baseline across epochs. Our experiments reveal that smaller models are more prone to extraction on fine-tuning data than bigger ones. Based on Figure 6.5, we can see that as the number of fine-tuning epochs increases, smaller models exhibit greater vulnerability to data extractability, whereas larger models demonstrate reduced exposure. Specifically, when attacking the StarCoder2-3B model after three epochs of fine-tuning, we extracted 36.8% additional samples relative to the baseline. In contrast, attacking the StarCoder2-15B model under the same conditions yielded 30.9% additional extractable samples compared to the baseline.

Table 6.5 and Figure 6.6 show the results of the experiments consisting of attacking the fine-tuned models controlling the duplication rate (i.e., the frequency of 300-token samples in the fine-tuning corpus). We find that the duplication rate has a notable impact on extractability. For StarCoder2-3B, extractability is below 5% with unique samples. This percentage increases to 23.7% with a single duplicate in the fine-tuning corpus and further to 45.1% with two duplicates.

Table 6.6 illustrates the effect of extending the input prefix length on the success of fine-tuned code extraction attacks. Our results indicate that the extractability of fine-

Table 6.5: Fine-tuning code attack on fine-tuned models with varying duplication rates.

Model	Duplication	Memorization rate			
		EM	BLEU	EM↑%	BLEU↑%
Starcoder2-3B	=1	0.049	0.423	0.025	0.074
	=2	0.237	0.615	0.172	0.174
	=3	0.451	0.754	0.329	0.333
	>3	0.631	0.764	0.368	0.266
Starcoder2-7B	=1	0.061	0.433	0.042	0.082
	=2	0.238	0.616	0.181	0.174
	=3	0.450	0.776	0.348	0.337
	>3	0.578	0.749	0.329	0.202
Starcoder2-15B	=1	0.075	0.475	0.052	0.110
	=2	0.352	0.694	0.277	0.244
	=3	0.544	0.832	0.417	0.385
	>3	0.609	0.774	0.309	0.198

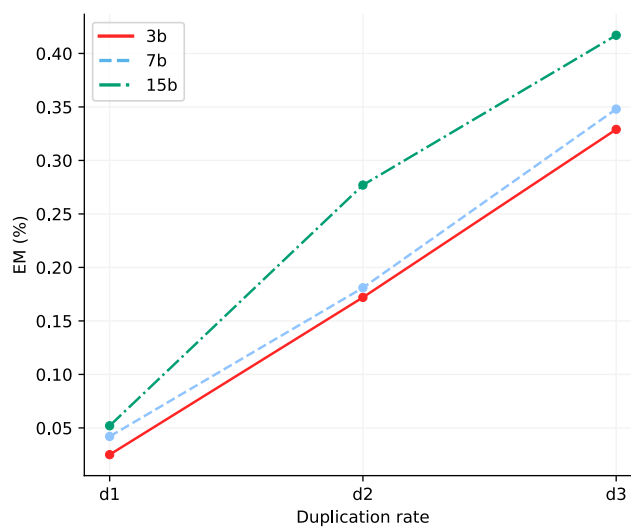


Figure 6.6: EM (↑%) when performing the fine-tuning code attack on fine-tuned models with varying duplication rates.

Table 6.6: Fine-tuning code attack on fine-tuned models with varying prefix lengths.

Model	Prefix length	Memorization rate			
		EM	BLEU	EM↑%	BLEU↑%
Starcoder2-3B	100 tokens	0.631	0.764	0.367	0.226
	150 tokens	0.689	0.785	0.393	0.231
	200 tokens	0.747	0.816	0.443	0.260
	250 tokens	0.815	0.865	0.502	0.297
Starcoder2-7B	100 tokens	0.578	0.750	0.330	0.202
	150 tokens	0.646	0.773	0.386	0.220
	200 tokens	0.712	0.802	0.436	0.243
	250 tokens	0.800	0.850	0.492	0.245
Starcoder2-15B	100 tokens	0.610	0.775	0.310	0.198
	150 tokens	0.730	0.805	0.372	0.216
	200 tokens	0.817	0.830	0.441	0.232
	250 tokens	0.888	0.868	0.474	0.244

tuning data increases with the input prefix length. We can extract 63.1% of the samples from StarCoder2-3B inputting 100 prefix tokens. This percentage increased to 68.9% with a prefix of 150 tokens and further to 81.5% with a prefix of 250 tokens.

In Figure 6.7, we present the percentage increase over the baseline in the exact match rate for different input prefix lengths across various model sizes. Our analysis reveals that smaller models remain more vulnerable to data extraction attacks. Especially, the difference in extractability between model sizes is more pronounced with an input prefix length of 100 tokens compared to lengths in the range of 150 to 250 tokens. These findings highlight that extending the input prefix length particularly impacts extractability, despite model size.

RQ3: Smaller models are more prone to extraction of fine-tuning data. Duplication rate significantly affects extractability. Increasing fine-tuning epochs leads to more samples memorized, and extending the input prefix length also facilitates fine-tuning extractability.

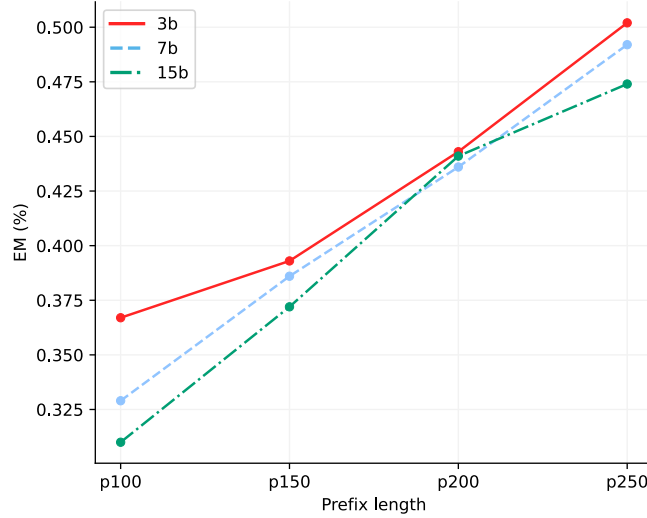


Figure 6.7: EM ($\uparrow\%$) when performing the fine-tuning code on fine-tuned models with varying prefix lengths.

6.4 RQ4: Type of Extracted Data

Table 6.7 represents the type of code data extracted through our attacks. During the tagging process, we find numerous examples of names, emails, and GitHub links memorized by the model [19].

Figure 6.8 shows the percentage per category of memorized samples performing the pre-training code attack on the base and fine-tuned models, respectively. The results indicate that *data carriers* and *licenses* are the most frequently memorized pre-training samples. Fine-tuning has an impact on the extractability of pre-training data. We find that this rate is not uniform between categories. Some categories are less extractable than others. Specifically, licenses become less extractable after fine-tuning compared to data carriers. For instance, the StarCoder2-3B base model memorizes 75.61% of the data carriers in the pre-training code attack dataset; this percentage decreases to 18.90% following fine-tuning. Similarly, the base model retains 68.55% of the licenses, but this figure drops to 6.45% after fine-tuning. Data carriers exhibit the highest memorization rate post-fine-tuning, being the category of code less affected by the “forgetting” phenomenon observed when performing the pre-training code attack on fine-tuned models.

Table 6.7: Code attack results per data category and model.

Code Attack type		Pre-training (RQ1-2)		Fine-tuning (RQ3)
Model	Category	Base	Fine-tuned	Fine-tuned
Starcoder2-3B	Code	115	53	56
	Dicts	124	31	54
	Docs	32	4	104
	License	85	8	185
	Test	3	2	1
Starcoder2-7B	Code	117	64	53
	Dicts	112	26	52
	Docs	42	5	123
	License	88	11	181
	Test	3	3	1
Starcoder2-15B	Code	136	62	88
	Dicts	133	47	64
	Docs	52	16	137
	License	94	15	195
	Test	4	4	1

The last column of Table 6.7 and Figure 6.9 present respectively the amount and the percentage of memorized samples per category from the fine-tuning code attack. The results indicate that similar to pre-training code attacks on pre-trained models, data carriers and licenses are the most memorized samples after fine-tuning. Specifically, the fine-tuned StarCoder2-15B model retained 54.70% of the data carriers and 68.66% of licenses from the fine-tuning code attack dataset.

RQ4: Data carriers and license information are the most likely to be memorized, while the latter is the most likely to be forgotten after fine-tuning.

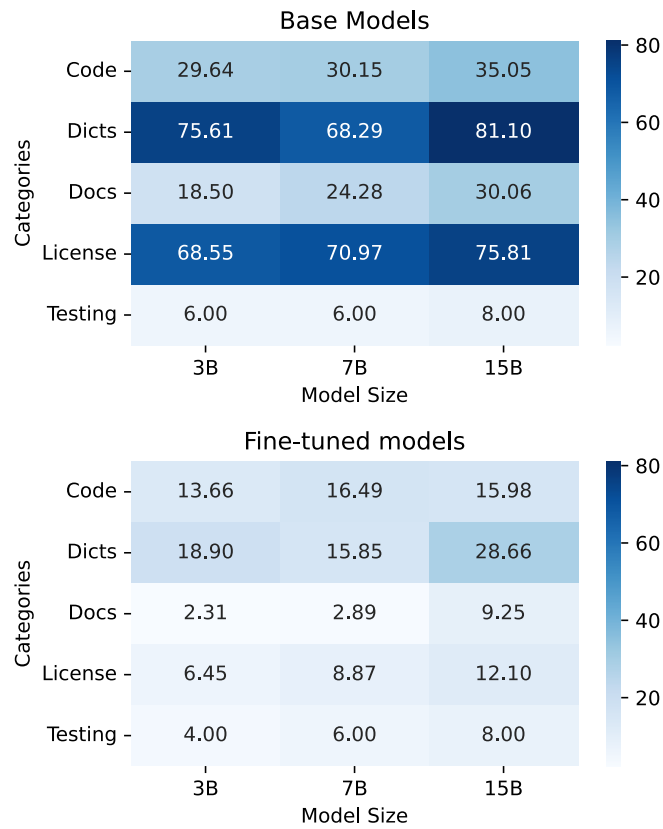


Figure 6.8: Percentage of extractable pre-training samples per category on base and fine-tuned models.

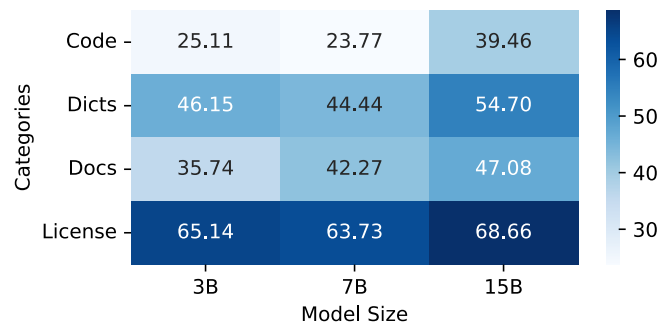


Figure 6.9: Percentage of extractable fine-tuning samples per category.

Chapter 7

Discussion

This chapter provides a comprehensive analysis of the obtained results. First, we will examine the findings concerning pre-training and fine-tuning memorization. Subsequently, we will discuss the broader implications of the study, along with relevant ethical considerations. Finally, the chapter will address the limitations of the work and potential threats to its validity.

7.1 Pre-train Data Memorization - Pre-trained Models

Our findings indicate that the StarCoder2 base model family tends to memorize its pre-training data. Consistent with prior research [5], [7], [84], we observed that the extent of memorization correlates with model size. Using a 100-token attack on the StarCoder2-15B model, we extracted 45.5% of the dataset. This rate dropped to 38.9% with the three-billion-parameter version. Our results substantiate the log-linear relation between size and memorization, which has been recently observed also for LLMs trained on code [5].

Our experiments reveal that extending the input prefix length enhances the memorization rate of the pre-trained models when attacking with samples from the pre-training data. These findings align with previous studies, which demonstrated that longer input prefixes facilitate the retrieval of more information from the model [7], [84]. Our results reinforce the connection identified by Carlini *et al.* between extractability and the precise

identification of key information necessary for accessing specific details from a language model [3].

Al-Kaswan *et al.* demonstrated that data carriers are memorized at a higher rate compared to regular code or documentation [5]. Our findings are consistent with this observation, as our categorization revealed that the types of code most frequently memorized by the pre-trained models were data carriers and licenses. We extracted up to 80% and 75% of the total data carriers and licenses from the attack dataset, but only 30% for code and documentation.

7.2 Pre-train Data Memorization - Fine-tuned Models

Performing the pre-training code attack on the fine-tuned models revealed that fine-tuning reduces the extractability of pre-training data. We observed that after three epochs of fine-tuning, around 30% of the samples that were previously extracted during the pre-training stage could no longer be extracted. To further investigate this phenomenon, we replicated the experiments conducted on the pre-trained models to examine how factors influencing pre-training extractability in pre-trained models affect fine-tuned models.

Our experiments revealed that the decrease in extractability is consistent across different model sizes to the same extent. Additional fine-tuning epochs do not significantly decrease extractability; despite some fluctuations, the EM rate does not consistently drop across epochs. For the 3B and 7B parameter models, extractability after the first epoch is slightly lower compared to later epochs. For example, the EM rate for the StarCoder2-3B model is 8.7% after the first fine-tuning epoch, increasing to 10% by the second epoch and remaining stable through the third epoch. This pattern likely reflects the model’s initial adjustment to new, unseen data during the first epoch of fine-tuning. The model requires a few epochs to adapt to this new data distribution and perform the code completion task effectively. The initial drop at the first epoch is more pronounced in the 7B model. Lozhkov *et al.* state that it is not clear why StarCoder2-7B is not performing as well as the 3B and 15B models for their size [8]. We considered this as a plausible explanation for

the observed discrepancies. Note that after the initial drop, extractability stabilizes in subsequent epochs.

Inspecting the type of pre-training code that is no longer extractable by fine-tuned models, showed that some categories are less extractable than others. Specifically, licenses become less extractable after fine-tuning compared to data carriers. StarCoder2-3B base model memorized 68.55% of the licenses, but this figure dropped to 6.45% after fine-tuning. Notably, data carriers exhibit the highest memorization rate post-fine-tuning, suggesting that this category of code is less affected by the “forgetting” phenomenon observed in the fine-tuned models.

7.3 Fine-tuning Data Memorization

Performing the fine-tuning code attack revealed that, even only one training epoch is sufficient to make the model memorize 55% of the attack dataset. This number goes up to 61% after the third epoch. Overall, extractability increases at each fine-tuning epoch.

Attacking different model sizes, allowed us to observe that smaller models memorize more fine-tuning data. As shown in Figure 6.5, this behavior happened only from the second fine-tuning epoch. Attacking the fine-tuned three billion model we were able to extract 36.8% of additional samples. This number decreased to 30.9% attacking the fine-tuned fifteen billion model.

Carlini *et al.*, Kandpal *et al.* revealed that the duplication rate severely impacts extractability. This result also persists in fine-tuning memorization. For StarCoder2-3B, extractability is below 5% with unique samples (i.e., $d = 1$). This percentage rises to 23.7% with a single duplicate in the fine-tuning corpus and further to 45.1% with two duplicates (i.e., $d = 3$).

Performing the fine-tuning code attack on the pre-trained models revealed that some fine-tuning samples were also retrievable from the pre-trained models. Several factors may contribute to this observation.

Notably, it indicates that even before fine-tuning, the pre-trained models had already memorized portions of the fine-tuning dataset, despite the extensive filtering process

applied to the fine-tuning data [98]. The repetitive and highly structured nature of Java syntax, which adheres to specific patterns that enhance readability, maintainability, and consistency, could be a significant factor. Additionally, our approach to constructing the attack dataset did not account for code complexity, leading to some suffixes being easily inferred by the model.

To further investigate, we inspected the types of code extractable from the pre-trained models. We found that the majority of the fine-tuning samples retrievable from the pre-trained models consisted of licenses. Specifically, performing the fine-tuning code attack on the StarCoder2-3B base model, we extracted 120 samples out of 917, of which 107 were licenses.

7.4 Implications

Risks for Fine-tuning Models: The study’s findings emphasize key implications for fine-tuning LLMs on code data. Firstly, models’ tendency to memorize training data poses significant risks, especially with proprietary or sensitive datasets. Fine-tuning on smaller, task-specific datasets increases exposure to memorization and data leakage. Smaller models are more prone to memorizing fine-tuning data compared to larger models. Although they are cheaper and faster to fine-tune, they carry a higher risk of data leakage. Therefore, companies and individuals fine-tuning code language models on personal or proprietary data should consider this risk when selecting the model size.

Security Vulnerability Propagation: LLMs may generate code snippets containing security flaws present in their training data, potentially introducing vulnerabilities into new software projects. This could lead to the unintentional spread of security issues across codebases [5], [7], [12].

Data Privacy Leakage Concern: If the model outputs verbatim snippets from proprietary or sensitive data used in fine-tuning, it could lead to unintended disclosure of confidential information. This is particularly problematic for industries dealing with personal, medical, or financial data. This also raises ethical questions about consent, especially if the data involves personal information or creative works used without explicit

permission for such reproduction [55].

Legal and IP Issues: Many existing LLMs for code utilize copyleft and other non-permissive licensed code [12]. Using public code to train these models can fall under fair use, which allows copyrighted works to be used in new and transformative ways in many jurisdictions [55]. However, if the model’s output closely resembles the copyrighted input, fair use may no longer apply. In such cases, the output must adhere to the original license terms, which may include share-alike and attribution requirements [55]. Outputting memorized samples from proprietary datasets could potentially infringe on intellectual property rights. This raises significant legal concerns, especially if the model is used commercially or made publicly accessible. Ensuring compliance with IP laws is crucial to avoid litigation and uphold ethical standards in model deployment [105].

Overfitting and Reduced Generalization: If the model memorizes specific examples from the fine-tuning data instead of learning general patterns, it may perform poorly on new, unseen data. This overfitting can limit the model’s effectiveness in real-world applications [3], [17].

7.5 Recommendations

To mitigate these risks, organizations that fine-tune foundation models on specialized or proprietary data need to implement robust safeguards. From a technical perspective, implementing differential privacy techniques like Whispered Tuning [96] can help ensure that the model does not memorize specific training data by adding noise during the training process. Additionally, using data filtering and deduplication methods can remove sensitive information and prevent the inclusion of duplicate data entries, thereby reducing the risk of memorization [84], [91], [92]. When fine-tuning models, it is important to limit the number of epochs, employ regularization techniques, and prefer larger models to prevent overfitting and memorization.

Organizational practices are also vital in ensuring the secure and ethical deployment of LLMs. Active monitoring of model outputs through automated checks is essential to detect and address any instances where the model might reproduce memorized data.

Developing methods to “forget” specific pieces of information post-training, such as unlearning techniques, can further safeguard against data leakage [5], [7].

Memorization can put creators and users of LLMs for code at legal risk, including license and fair use infringements [12], [55]. To mitigate these risks, it is crucial to train models using code licensed under permissive licenses (such as BSD-3 or MIT) or to provide provenance information that traces the code back to its source [5], [7], [12], [55].

7.6 Ethical considerations

Although this work explores techniques that could potentially be used to extract sensitive information from models, it is conducted with ethical considerations in mind. Our objective is to highlight the issue of memorization in LLMs for code, inform users and creators about this problem, and provide tools for measuring it. We emphasize that our work does not intentionally expose private information and we strongly urge users of our work to avoid such practices. We believe the benefits of sharing our data outweigh the risks, and we decide to make them available.

7.7 Limitations and Threats to the Validity

Internal Validity: In our evaluation, we did not account for the spatial distribution of samples within the files. While the samples maintain a fixed token length, their locations within the files are selected arbitrarily to avoid bias. The methodology was chosen to ensure samples could be extracted from any location within the file to reflect a realistic and varied scenario [5], [84].

External Validity: While we study three model sizes, our evaluation focuses on the StarCoder2 model family. Other model architectures may exhibit different levels of memorization. Additionally, our pre-training code dataset considers samples that are duplicated more than three times in the training corpus. Due to computational constraints, we were not able to control the duplication rate as we did on the fine-tuning code attack dataset. More studies on pre-training code attacks on low-duplication data could

extend our findings. Most previous studies have centered on Python. This leaves a gap for other programming languages. We focused on Java for our attack datasets due to its well-defined syntax and structure, which may impact memorization. Given Java’s popularity, our results may not apply to less common languages. In future evaluations, we plan to include more programming languages to compare how different languages are memorized by the model.

Construct Validity: We primarily use the EM metric for reporting extractability rates in code LMs as we focus on exact reproductions to address privacy and security concerns associated with memorization. However, this metric likely underestimates the true extent of memorized samples, as some might be slightly altered by the model [17]. To mitigate this threat, we also compute and report other fuzzy metrics such as BLEU, METEOR, and ROUGE-L in the paper and in our replication package [19]. Our findings indicate that these metrics are highly correlated with the EM rate. However, even these metrics might be biased, as a high degree of alteration does not necessarily imply that the model did not memorize the original content. We specifically focus on verbatim memorization and use extractability as a proxy for this purpose. Previous research has indicated that models not only memorize exact text but also ideas and writing styles, further complicating the assessment of true memorization [17].

Chapter 8

Conclusions

In this study, we conducted an extensive investigation into data extraction from pre-trained and fine-tuned code language models. We formally defined a data extraction security game based on the concepts of k-extractability and membership inference attacks. This framework was employed to develop a custom benchmark for assessing the vulnerability of both pre-training and fine-tuning samples to extraction attacks. Our findings indicate that the StarCoder2 model memorizes its pre-training data. We further observed that fine-tuning reduces the extractability of pre-training data. Contrarily, we found that fine-tuning increases the vulnerability of smaller models to fine-tuning data extraction attacks compared to larger models. Additionally, our results show that data carriers and license information are the most likely to be memorized, with the latter being the most likely to be forgotten after fine-tuning. These insights emphasize the importance of addressing memorization during fine-tuning to safeguard against potential data leaks and highlight the need for further research and development of strategies to mitigate these vulnerabilities.